CSAEC510 · PROMPT ENGINEERING · 5TH SEMESTER

Prompt Engineering CIE-1 Study Guide

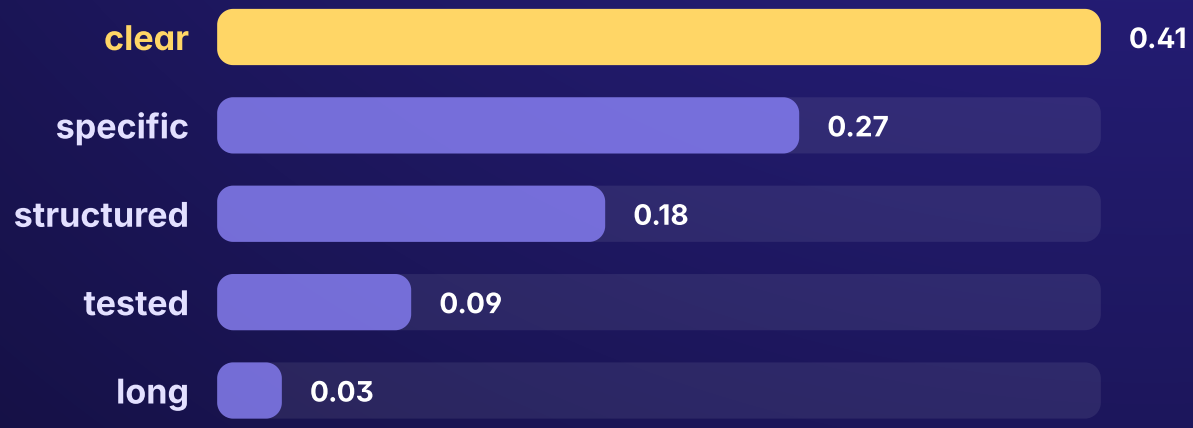
Everything for your first internal test, explained from zero, with diagrams, highlighted key points, exam tips and likely questions with answer plans.

Unit I · Five Principles & LLMs

Unit II · Standard Practices

Unit III · LangChain → Chat Models

A good prompt is ____



↑ how an LLM "thinks": it scores every possible next token and picks one (you'll understand this by page 5)

Built from your 4 class decks (Session 1, Module 1 Session 2, Module 2 Sessions 1 & 2 — 192 slides)
and the textbook *Prompt Engineering for Generative AI*, James Phoenix & Mike Taylor (O'Reilly, 2024)

START HERE

0 How to Use This Guide

This guide covers your **entire CIE-1 portion**: Unit I, Unit II and Unit III up to **Chat Models**. Every slide from your class decks is included, rewritten in simple language, with the textbook filling the gaps. The coloured boxes tell you what each piece of text is for:

KEY IDEA
The core definition or idea. **Learn these word-for-word** — they make perfect opening lines.

EXAM TIP
How examiners ask about this topic, and how to score full marks.

REMEMBER
Short summaries and memory tricks to lock the idea in.

WATCH OUT
Common mistakes and traps. Read these twice.

EASY ANALOGY
Everyday comparisons that make hard ideas click.

BEYOND YOUR SLIDES
Extra material from the textbook that your slides only list, or that adds depth.

Prompt / Output boxes show real prompts: **PROMPT** is what you type, **OUTPUT** is what the AI replies. A red label means a bad prompt and a green label means a good one. Yellow highlights mark the phrases worth remembering.

Study plan — about 7 hours in 6 sittings

	Sitting	Topics	Time
☐	1	1.1 – 1.5 · Prompt engineering, next-token prediction, five principles overview	75 min
☐	2	1.6 – 1.10 · The five principles in depth	75 min
☐	3	1.11 – 1.19 · LLMs, tokens, vectors, Transformer, probabilities · R1	75 min
☐	4	2.1 – 2.9 · Lists, regex parsing, JSON, YAML, diverse formats	75 min
☐	5	2.10 – 2.22 · Style, chunking, role prompting, hallucinations · R2	75 min
☐	6	3.1 – 3.5 · LangChain, modules, setup, chat models · R3 + cheat sheet	45 min

FAST TRACK
Only have 2 hours? Read just the **Key idea**, **Remember** and **Exam tip** boxes and look at every diagram. Then do the three revision pages (R1, R2, R3) and the cheat sheet.

REVISION ROUTINE
Night before: cheat sheet + "Likely CIE questions" (R1–R3). Draw from memory: LLM pipeline, self-attention, RLHF stages, YAML filter chart, chunking table.

Contents

UNIT I — FIVE PRINCIPLES & LLM FOUNDATIONS

1.1 What is Prompt Engineering? 6
1.2 The Naive Prompt and its 5 Problems 6
1.3 Why Prompts Matter: How the AI Picks the Next Token 8
1.4 Improving a Prompt: The Engineered Prompt 10
1.5 Overview of the Five Principles 12
1.6 Principle 1 — Give Direction 12
1.7 Principle 2 — Specify Format 14
1.8 Principle 3 — Provide Examples 15
1.9 Principle 4 — Evaluate Quality 16
1.10 Principle 5 — Divide Labor 19
1.11 What Are Large Language Models (LLMs)? 22
1.12 What Are Text Generation Models? 22
1.13 Tokens and Tokenization 22
1.14 Pretraining and Fine-Tuning 24
1.15 Vector Representations: The Numerical Essence of Language 25
1.16 Transformer Architecture: Orchestrating Contextual Relationships 26
1.17 Probabilistic Text Generation: The Decision Mechanism 28
1.18 Parameters, Compute, Tensors and Data Quality 30
1.19 Historical Underpinnings and Model Families 31
R1 Unit I — Quick Revision & Likely Questions 35

UNIT II — STANDARD PRACTICES FOR TEXT GENERATION

2.1 Generating Lists 39
2.2 Hierarchical List Generation 40
2.3 Parsing with Regular Expressions — Example 3-1 42
2.4 Keeping the Structure — Example 3-2 (Dictionary) 44
2.5 When to Avoid Using Regular Expressions 46
2.6 Generating JSON 46
2.7 YAML 47
2.8 Filtering YAML Payloads 48
2.9 Diverse Format Generation with ChatGPT 49
2.10 Explain It Like I'm Five (ELI5) 50
2.11 Universal Translation Through LLMs 50
2.12 Ask for Context 51
2.13 Text Style Unbundling 52
2.14 Identifying the Desired Textual Features 52
2.15 Generating New Content with the Extracted Features 53
2.16 Extracting Specific Textual Features with LLMs 53
2.17 Chunking Text and Strategies 54

2.18 Role Prompting 56
2.19 GPT Prompting Tactics 57
2.20 Avoiding Hallucinations with Reference 57
2.21 Give GPT "Thinking Time" and Inner Monologue 59
2.22 Self-Eval LLM Responses 60
R2 Unit II — Quick Revision & Likely Questions 61

UNIT III — LANGCHAIN (CIE-1 PORTION)

3.1 Why Basic Prompting Is Sometimes Not Enough 65
3.2 Introduction to LangChain 65
3.3 Six Common Modules of LangChain 66
3.4 Environment Setup for LangChain 67
3.5 Chat Models 69
R3 Unit III (CIE-1 part) — Quick Revision & Likely Questions 70

FINAL REVISION

★ One-Page Cheat Sheet 72
A–Z Glossary 73
✔ Exam-Day Answer-Writing Tips 74

UNIT I

The Five Principles of Prompting & Introduction to LLMs for Text Generation

Part A teaches you how to write prompts that give reliable results. Part B opens up the model: how text becomes numbers, how the Transformer understands context, and how the next word is chosen.

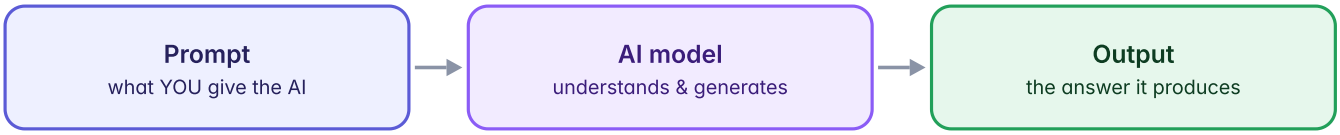
What is Prompt Engineering?	1.1	The Naive Prompt and its 5 Problems	1.2
Why Prompts Matter: How the AI Picks the Next Token	1.3	Improving a Prompt: The Engineered Prompt	1.4
Overview of the Five Principles	1.5	Principle 1 — Give Direction	1.6
Principle 2 — Specify Format	1.7	Principle 3 — Provide Examples	1.8
Principle 4 — Evaluate Quality	1.9	Principle 5 — Divide Labor	1.10
What Are Large Language Models (LLMs)?	1.11	What Are Text Generation Models?	1.12
Tokens and Tokenization	1.13	Pretraining and Fine-Tuning	1.14
Vector Representations: The Numerical Essence of Language	1.15	Transformer Architecture: Orchestrating Contextual Relationships	1.16
Probabilistic Text Generation: The Decision Mechanism	1.17	Parameters, Compute, Tensors and Data Quality	1.18
Historical Underpinnings and Model Families	1.19	Unit I — Quick Revision & Likely Questions	R1

⏱ ~3.5 hours · Sittings 1–3

1.1 What is Prompt Engineering?

KEY IDEA

Prompt engineering = writing the right instructions for an AI so that you get the answer you want. It is the process of **designing and improving** a prompt to get **better, more useful and more accurate** results.



The basic flow. Easy formula: Prompt → AI Model → Output, and Better Prompt → Better Output.

Simple example — naming a shoe

Suppose you want the AI to suggest names for a shoe that can fit **everyone**.

PROMPT

```
Give me a list of names for shoes that can fit any foot size.
```

OUTPUT

```
UniFit SoleStrides
FlexiSize Shoes
OmniFit FootFlex
AnyStep Walkers
UniversalSteps
```

Why is it important?

A **poor prompt** may give a vague answer. A **better prompt** gives the AI more specific instructions, so the output is more useful.

POOR

```
Tell me about shoes.
```

BETTER

```
Give me 10 creative names for
comfortable shoes that can fit
different foot sizes.
```

REMEMBER

In one line: Prompt Engineering = giving clear and effective instructions to AI to get the desired output.

1.2 The Naive Prompt and its 5 Problems

A **naive prompt** means we simply ask the AI what we want **without giving many instructions**, e.g. *"Give me names for shoes that fit any foot size."* The AI can still produce good names, and for **one-time, casual use** this may be enough. But if we are building a **real product used by thousands of people**, we need a much better prompt.

Problem	Easy meaning	Example	Fix (principle)
1. Vague direction	We haven't told the AI what kind of answer we want	Should the name be short? Modern? Funny? Professional?	Give Direction
2. Unformatted output	AI may give the answer in a different format each time	Sometimes numbered, sometimes with explanations	Specify Format
3. Missing examples	We haven't shown the AI examples of good answers	Give examples like Nike, Adidas, Puma, Reebok	Provide Examples
4. Limited evaluation	No way to measure whether the answers are good	How do we decide which name is best?	Evaluate Quality
5. No task division	We ask the AI to do everything at once	Generate → evaluate → improve, all in one step	Divide Labor

EXAM TIP

Notice the **one-to-one mapping**: each of the 5 problems is solved by one of the 5 principles. If asked "What are the limitations of a naive prompt?", list the 5 problems **and** name the principle that fixes each — easy full marks.

How each problem looks in practice

1. Vague direction

VAGUE

```
Give me shoe names.
```

CLEAR

```
Generate 10 short, modern, easy-to-pronounce shoe brand names. Use one or two words and make them suitable for a global market.
```

"Give me shoe names" could produce *anything*. The second prompt gives clear instructions.

2. Unformatted output

Ask *"Give me 10 shoe names"* and one time the AI says *"Here are some great names: FlexStep, UniFit, WalkPro..."*, another time it responds differently. That is a problem if a **computer program** needs to read the answer automatically. Fix: *"Return only the names, one name per line. Do not include numbers or explanations."* → now the output is **consistent**.

3. Missing examples

PROMPT

```
Examples of good names: Nike, Adidas, Puma, Reebok.
Generate 10 new names with a similar short and memorable style.
```

Examples show the AI the **style** we want. Providing examples is called Few-Shot Prompting.

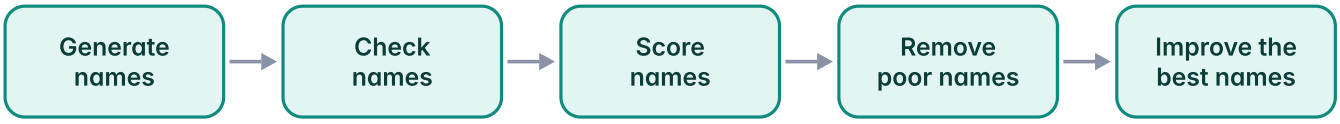
4. Limited evaluation

If the AI gives us 100 names, how do we know which are good? Define **criteria with weights** and score every name:

Criterion	Weight
Easy to remember	30%
Easy to pronounce	20%
Unique	20%
Suitable for global customers	20%
Short	10%

Now we can **score** the generated names, **compare** different prompts and **improve** them.

5. No task division



Instead of one complicated job, divide the work. This is called task decomposition.

REMEMBER

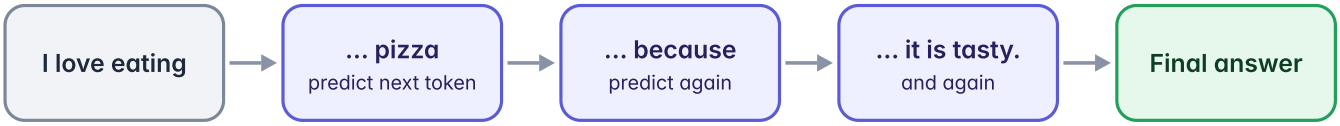
Simple prompt → good enough for casual use.
Engineered prompt → better, consistent, measurable results for real applications.
A basic prompt says *"Give me some shoe names."* Prompt engineering says *"Give me 10 short, modern, memorable shoe names. Use one or two words. Follow these examples. Return only the names. Score each name based on these criteria."*

1.3 Why Prompts Matter: How the AI Picks the Next Token

KEY IDEA

An AI does **not** think of the complete answer at once. It generates the answer one token at a time, each time predicting **what should come next**.

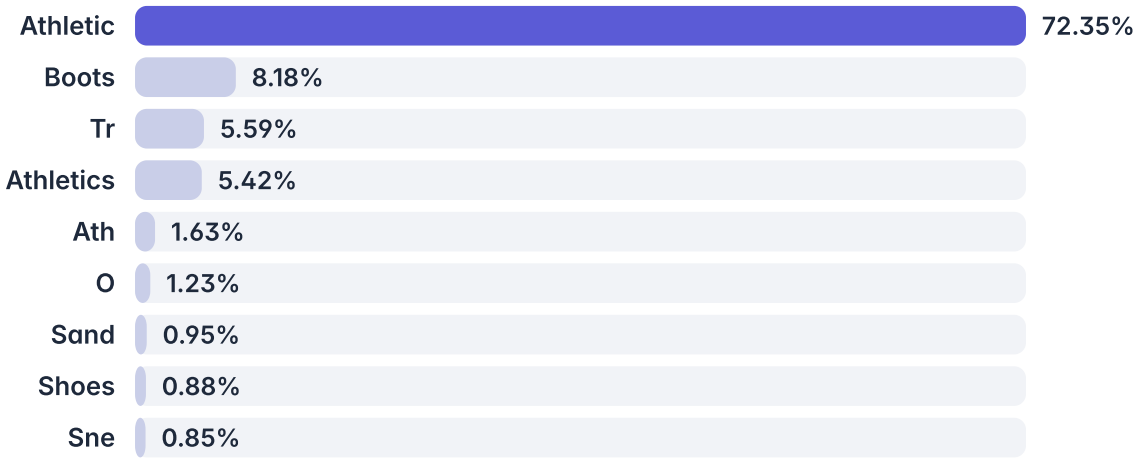
A **token** is a small piece of text — a complete word, part of a word, or sometimes punctuation.



Prompt → predict next token → predict next token → predict next token → final answer

The prompt is the starting context

Given the text `AnyFit ____`, the AI considers many possible next words, each with a **probability**. This is the real breakdown from the textbook (Figure 1-1, OpenAI Playground):

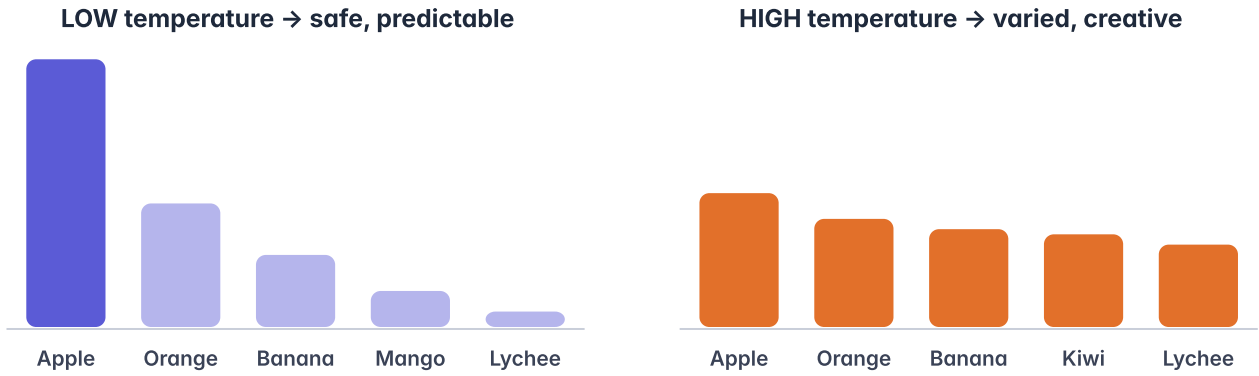


Next-token probabilities after "AnyFit". Because "Athletic" is far more probable, the AI is more likely to write "AnyFit Athletic" than "AnyFit Shoes".

Notice the odd tokens like `Tr`, `Ath`, `Sne` — these are **parts of words** (Tr-ainers, Ath-letic, Sne-akers). This proves that tokens are not always whole words.

What is randomness? (temperature)

The AI doesn't *always* choose the highest-probability word. There is an element of **randomness**, controlled by a setting called temperature.



Bar height = probability of being picked next. Higher temperature flattens the bars, so less-likely words get chosen more often.

Low temperature → Apple, Orange, Banana, Mango. Higher temperature → Dragon fruit, Star apple, Kiwi, Lychee...

Temperature	Behaviour	Use it for
Low (e.g. 0 – 0.3)	More predictable, consistent answers	Facts, data extraction, JSON, code
High (e.g. 0.8 – 1+)	More varied, creative answers	Brainstorming names, stories, ideas

Why does a small prompt change matter?

PROMPT 1 `Give me a shoe name.`

PROMPT 2
```
Give me a creative, premium,
athletic shoe brand name.
```

The context is different, so the **probabilities of the next tokens change**. Therefore the AI may produce completely different answers.

EASY ANALOGY

Think of AI like **autocomplete on your phone**. Type *"I am going to..."* and it may predict **"school"**. Type *"I am going to the airport to..."* and it may predict **"travel"**. The words you already typed influence what comes next.

MAIN IDEA

LLMs generate text by predicting **one token at a time** based on the prompt and the tokens already generated. Small changes in the prompt change these probabilities, which can significantly change the final output.

1.4 Improving a Prompt: The Engineered Prompt

AI models are trained to give helpful answers, but an **average or unclear prompt** gives an **average answer**. The words in the prompt change the probability of the next tokens — so Better prompt → Better and more reliable output.

ENGINEERE

```
Brainstorm a list of product names for a shoe that fits any foot size,
in the style of Steve Jobs.

Return the results as a comma-separated list, in this format:
Product description: A shoe that fits any foot size
Product names: [list of 3 product names]

## Examples
Product description: A refrigerator that dispenses beer
Product names: iBarFridge, iFridgeBeer, iDrinkBeerFridge

Product description: A watch that can tell accurate time in space
Product names: iNaut, iSpace, iTime

Product description: A home milkshake maker
Product names: iShake, iSmoothie, iShake Mini
```

OUTPUT

```
Product description: A shoe that fits any foot size
Product names: iFitFoot, iPerfectFit, iShoeSize
```

The 4 techniques used in this prompt

1 · Role / style
"in the style of Steve Jobs" → names that are simple, memorable, modern, easy to understand. Called **style prompting** or **persona/style instruction**.

2 · Output format
"Return the results as a comma-separated list" → `iFitFoot, iPerfectFit, iShoeSize` instead of one per line. Useful when a program reads the output.

3 · Examples
Refrigerator → iBarFridge..., Watch → iNaut..., Milkshake maker → iShake... The AI learns the pattern and applies it. This is **Few-Shot Prompting**.

4 · Clear task
"Brainstorm product names for a shoe that fits any foot size" → the AI knows **what to do** (generate names), **the product** (shoe) and **what is special** (fits any size).

	Simple prompt	Engineered prompt
Prompt	Give me names for a shoe that fits any foot size.	Brainstorm 3 product names... in the style of Steve Jobs. Return as a comma-separated list. Here are some examples...
Problem / result	AI has to **guess** the style, number of names, format...	AI knows **Task + Style + Format + Examples** → result is more predictable

What does "optimized prompt" mean?

A prompt improved so that it gives:

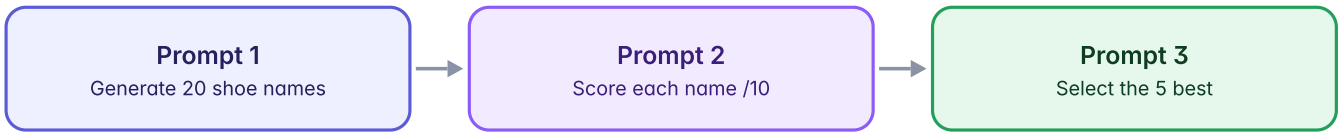
Good quality
useful answers

Reliability
similar format / results each time

Lower cost
avoids unnecessary tokens

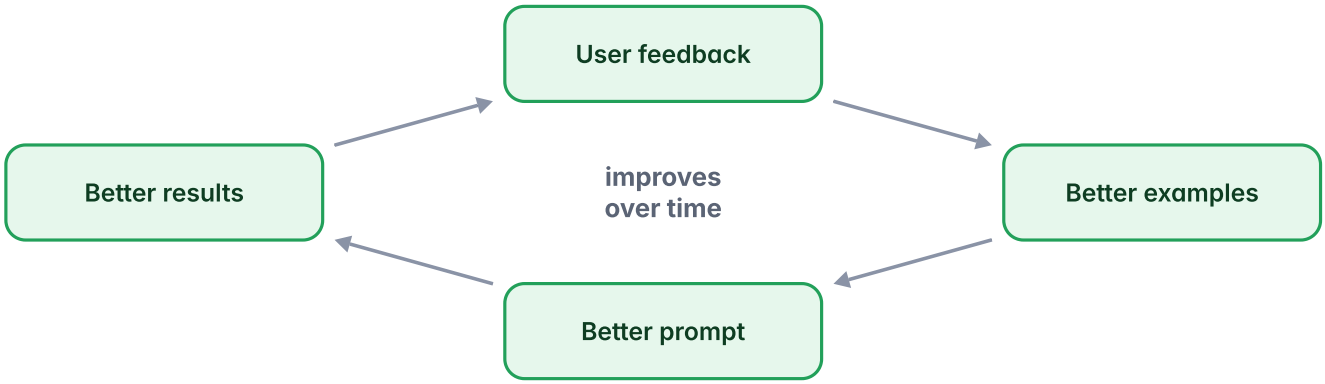
Consistency
follows instructions properly

A prompt can be part of a system (prompt pipeline)



The output from one prompt becomes the input to the next — a prompt pipeline.

The system can also **improve over time**: if users keep choosing *iFit, FlexFit, UniFit* and reject *FootMaster, ShoeWorld*, we use this feedback to update the examples in our prompt.



User feedback → Better examples → Better prompt → Better results

Good Prompt = Clear Task + Style + Format + Examples + Constraints

REMEMBER

Prompt Engineering is **not just asking AI a question**. It is carefully designing the instructions so that the AI produces high-quality, consistent and useful results.

1.5 Overview of the Five Principles

KEY IDEA

The textbook's five principles of prompting are **timeless** — they work across models (GPT, Claude, Gemini...) and even for image models. They answer one question: *how do I get reliable output from an AI?*

1
Give Direction
Tell the AI clearly **what** you want (style, persona, rules).

2
Specify Format
Tell the AI **how** the answer should look (list, JSON...).

3
Provide Examples
Show the AI what good answers look like.

4
Evaluate Quality
Check & measure whether the answer is good.

5
Divide Labor
Break a difficult task into smaller steps.

MEMORY TRICK

Direction → Format → Examples → Evaluate → Divide
Direction = **WHAT** you want · Format = **HOW** it is presented · Examples = **SHOW** what good looks like · Evaluation = **CHECK** how good it is · Divide Labor = **BREAK** big tasks into small tasks

REAL-LIFE ANALOGY (USE THIS IN YOUR ANSWER!)

Imagine you ask a student to prepare an answer:
Give Direction — "Prepare a 5-mark answer on AI."
Specify Format — "Give it in 5 bullet points."
Provide Examples — "Here is an example of a good 5-mark answer."
Evaluate Quality — "Check the answer using a 5-mark rubric."
Divide Labor — "First research → then write → then check → then improve."
That is essentially Prompt Engineering.

1.6 Principle 1 — Give Direction

KEY IDEA

Give Direction = tell the AI exactly what kind of result you want — describe the desired style in detail, or reference a relevant persona.

For *"Give me a name for a shoe"*, the AI doesn't know: should it be modern? funny? professional? short? easy to remember? for children? for sports? So we give more direction:

DIRECTED `Give me a short, modern and memorable name for a shoe that can fit any foot size.`

Ways to give direction

① Use a persona

PERSONA	`Generate a shoe name in the style of Steve Jobs.`
OUTPUT	`Simple, short, memorable names`

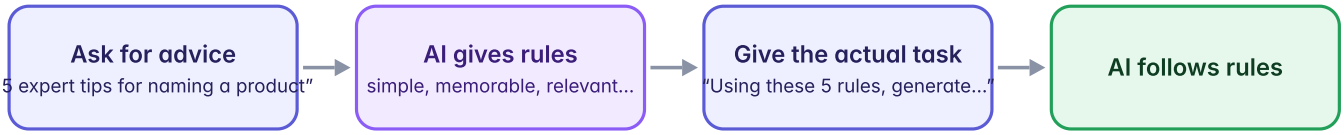
PERSONA	`Generate a shoe name in the style of Elon Musk.`
OUTPUT	`ShoeX, MarsFit, HyperShoe`

Changing the persona can change the output significantly: Steve Jobs style → simple/minimal; Elon Musk style → futuristic/technology-oriented.

② Give the AI rules

The product name should: be short · be easy to pronounce · be easy to remember · have a positive meaning · be suitable for international customers. Now the AI has **specific criteria** to follow.

③ Ask the AI for advice first (AI generates its own direction)



The AI uses its own previous answer as guidance.

AI might reply: keep the name simple · make it memorable · make it relevant · make it unique · give it a positive meaning.

④ Use external expert advice

Instead of asking AI, provide advice from a **trusted source**: *"Follow these five expert rules for product naming: easy to pronounce, easy to spell, easy to remember, unique, creates a positive emotion. Generate 20 shoe names following these rules."* This gives the AI additional **knowledge and constraints**.

DISADVANTAGE

A **longer prompt contains more tokens** → higher cost + more processing. Prompt engineering is about finding the right balance between Quality + Reliability + Cost.

Direction in image generation

WEAK	`People in a business meeting.`

BETTER	`Four people in a business meeting, sitting around a glass-top table, looking at a white laptop, realistic stock photography.`

The better prompt gives direction about: **number of people · location · table · laptop · photography style**. The result becomes more specific.

- **Changing one detail can change the image** — replace "glass-top table" with "a campfire in the woods" and the generated image becomes completely different. Even a small change in the prompt can significantly change the output.
- **Using an artist's style** — "People in a business meeting, by Van Gogh" (or "Van Gogh-inspired") → expressive brushwork and vivid colours. This is extra **style direction**.

Don't give too many instructions

"More instructions = better answer" is **not always true**. Asking for an image that is Van Gogh style + realistic stock photo + futuristic + cartoon + black and white + highly detailed + a specific camera + outdoors + indoors gives **conflicting** instructions — the AI doesn't know which matters most.



The goal is clear and relevant direction.

1.7 Principle 2 — Specify Format

KEY IDEA

Specify Format = tell the AI exactly how you want the answer presented — define what rules to follow and the required structure of the response.

The same information can come as a **paragraph · numbered list · table · JSON · CSV · Python code · YAML · comma-separated list**. If we don't specify, the AI may choose a **different format each time**:

PROMPT
```
Give me 5 shoe names.
```

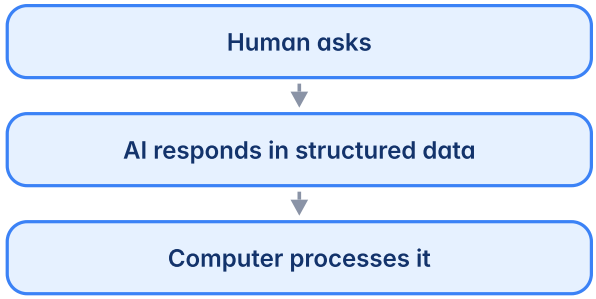
OUTPUT
```
Run 1:  FlexFit
        OmniFit
        AdaptStep
Run 2:  FlexFit, OmniFit, AdaptStep
Run 3:  Here are five possible names: ...
```

Fine for a person reading it — a **problem for software**. If your program expects `FlexFit, OmniFit, AdaptStep` but gets `1. FlexFit 2. OmniFit 3. AdaptStep`, it may not know how to process it. So we say *"Return only a comma-separated list."* — or, even better for software, *"Return only JSON."*

RETURN ONLY JSON

```
{
  "product_description": "A shoe that fits
any foot size",
  "product_names": [
    "FlexFit",
    "OneSize Step",
    "Adapt-a-Shoe"
  ]
}
```

Why JSON? JSON is a structured format that computers can easily understand:



Product name: FlexFit, Price: 999, Category: Shoes → `{"product_name": "FlexFit", "price": 999, "category": "Shoes"}` — very useful in software applications and APIs.

Format for image generation

"Business meeting" is very general. "Business meeting **in Minecraft**" specifies the visual format. Other formats: **stock photo · oil painting · cartoon · pencil sketch · watercolor · Minecraft · 3D render**. For images, format tells the AI what **type of visual output** you want.

Direction vs Format — an important distinction

	Direction	Format
Tells the AI...	what **style or characteristics** you want	what **type / form** of output you want
Question it answers	**How should it look / behave?**	**What form should the output take?**
Example	Create a business meeting *in the style of Van Gogh*.	Create a business meeting *as a realistic stock photo*.

EXAM TIP

"Differentiate between Give Direction and Specify Format" is a classic short question. Write the one-line difference + one example each.

1.8 Principle 3 — Provide Examples

KEY IDEA

Provide Examples = show the AI examples of what a good answer looks like (insert diverse test cases where the task was done correctly).

EASY ANALOGY

Training a new employee: instead of only saying "Do this task", you would show them a few examples. The same idea works with AI.

Zero-shot = no examples

`Generate 3 names for a shoe that fits all foot sizes.`
The AI has to figure out what you want by itself.

One-shot = one example

`Example: Milkshake maker → HomeShaker, QuickShake. Now generate names for a universal shoe.`
AI learns the pattern from one example.

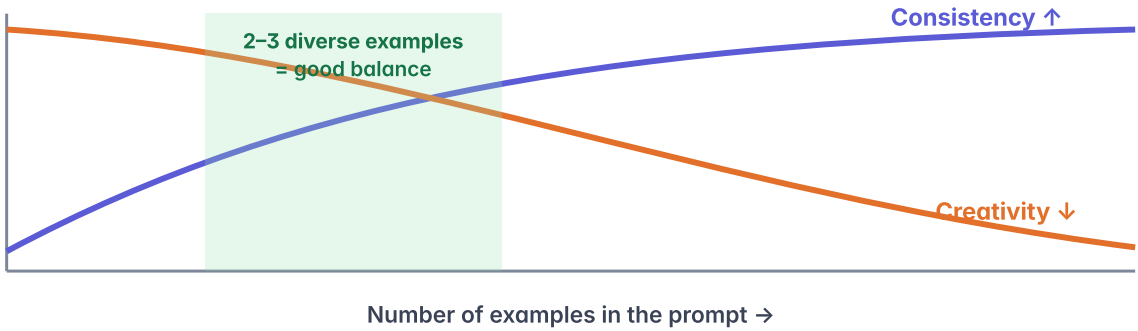
Few-shot = several examples

`Milkshake maker → HomeShaker, QuickShake · Space watch → AstroTime, SpaceGuard · Beer refrigerator → BearFridge, CoolCat. Now generate…`
Clearer understanding of the pattern.

Why do examples help?

If your examples contain **animal names** and you say *"Generate animal-based shoe names"*, the AI may generate **FlexiFox, PandaPaws, ChameleonStep** — examples guide the AI toward the pattern you want.

But too many examples can be a problem



More examples → more consistency, but less creativity. 2–3 diverse examples: AI understands the pattern but stays creative. 20 very similar examples: AI may simply copy them.

REMEMBER

Examples should be useful and diverse. More examples ≠ always better.

1.9 Principle 4 — Evaluate Quality

KEY IDEA

Evaluate Quality = test the AI's answers and measure how good they are — identify errors and rate responses to find what drives performance.

Initially we might just look at the answer and say *"Looks good."* This is called **trial and error** or blind prompting. For a real AI application we need something more **systematic**.

What is evaluation (an "eval")?

Evaluation (Eval) = a systematic way of measuring how well an AI model or prompt performs — e.g. **accuracy, relevance, correct format, quality, consistency, user satisfaction**.

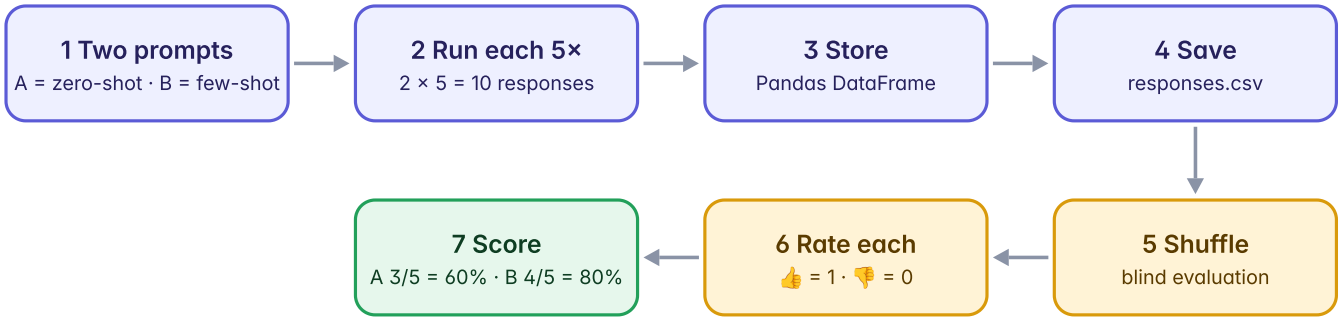
Simple thumbs-up / thumbs-down method (A/B testing)

We don't simply *assume* Prompt B (with examples) is better than Prompt A — we **test both**, e.g. run each 5 times and rate every response 👍 Good or 👎 Bad.

Prompt	Test 1	Test 2	Test 3	Test 4	Test 5	Score
A — *Generate shoe names.*	👍	👍	👎	👍	👎	3/5 = **60%**
B — *...using three examples, as a comma-separated list.*	👍	👍	👍	👍	👎	4/5 = **80%**

Prompt B is better **based on this test**. This is a simple form of A/B testing. (Another run in your slides gave A = 1/5 = 20% and B = 3/5 = 60% — same conclusion.)

The 7-step evaluation process (from the textbook notebook)



How the book compares a zero-shot prompt against a few-shot prompt.

1. **Create two prompts** — Prompt A = zero-shot (no examples); Prompt B = few-shot (contains examples).
2. **Run each prompt multiple times** — `num_tests = 5`, so 2 prompts × 5 tests = **10 responses**.
3. **Store the responses** in a **Pandas DataFrame** (think of it as an Excel table: Variant · Prompt · Response).
4. **Save to CSV** — `responses.csv`, so it can be opened in Excel.
5. **Shuffle the responses** — if you know which prompt produced a response, you might **unconsciously favour** it. Randomising makes it fair.
6. **Give a rating** — one response at a time: 👍 Good = **1**, 👎 Bad = **0**.
7. **Calculate the score** — A: 3 good out of 5 = 60%; B: 4 good out of 5 = 80% → Prompt B performs better.

SIMPLIFIED VERSION OF THE A/B TEST CODE

```
import pandas as pd

prompt_A = "Product description: A pair of shoes that can fit any foot size.\nProduct names:"
prompt_B = """Product description: A home milkshake maker
Product names: HomeShaker, QuickShake
...(more examples)...
Product description: A pair of shoes that can fit any foot size.
Product names:"""

num_tests = 5
rows = []
for variant, prompt in [("A", prompt_A), ("B", prompt_B)]:
    for _ in range(num_tests):                    # 2 prompts x 5 = 10 responses
        rows.append({"variant": variant, "prompt": prompt,
                     "response": get_response(prompt)})   # call the LLM

df = pd.DataFrame(rows)
df.to_csv("responses.csv", index=False)           # save for Excel
df = df.sample(frac=1)                            # shuffle -> blind evaluation
# a human rates each response: 1 = thumbs up, 0 = thumbs down -> df["feedback"]
print(df.groupby("variant")["feedback"].mean())   # e.g. A 0.6, B 0.8
```

Blind evaluation — why shuffle?

Response → randomly shown → you rate it. You **don't know which prompt produced it**, so you give an unbiased rating. This is called **blind evaluation**.

Why test multiple times?

AI does not always give exactly the same answer (Run 1: FlexFit, Run 2: OmniFit, Run 3: AdaptStep). Testing once can be misleading. **Even 5–10 tests** can reveal problems you wouldn't notice from a single test.

WHY EVALUATION IS IMPORTANT

You test once, say "Wow! Perfect!", and put it into production. After 1,000 users you discover: sometimes the **format is wrong**, sometimes **poor answers**, sometimes it **makes up information**, sometimes it **refuses unnecessarily**, sometimes **unsafe content**. Evaluation helps find these problems before production.

Human evaluation vs automatic evaluation

Human evaluation	Automatic evaluation
A **person** looks at the answer: 👍 / 👎 or a score like 8/10.	A **computer** evaluates the result, e.g. Expected = "Healthy", AI = "Healthy" → ✓ correct; AI = "Anthracnose" → × wrong.
Can be very accurate, but **slow + expensive** if thousands of responses must be evaluated.	**Much faster** for large-scale testing.

Different things we can evaluate (11 criteria)

#	Criterion	What it checks	Example
1	Cost	How much the AI call costs	Long prompt → more tokens → higher cost
2	Latency	How long the AI takes to respond	Prompt A 1 s vs Prompt B 10 s → A for real-time apps
3	Number of calls	How many AI calls one task needs	Generate → Check → Improve → Finalize = 4 calls (more time + cost)
4	Performance	Does the AI actually achieve the task?	95 of 100 diseases correct → accuracy 95%
5	Classification	Does it assign the correct category?	Image → Healthy / Anthracnose / Rust vs true label
6	Reasoning	Logical / mathematical mistakes	25 × 4 = ? AI says 120 instead of 100 → failed
7	Hallucination	Info that is not true / not supported	"Who invented this fictional technology?" → AI invents a person
8	Safety	Harmful, dangerous, inappropriate content	—
9	Refusals	Refusing a perfectly reasonable request	Refuses "Explain photosynthesis."
10	Adversarial testing	Deliberately trying to break the system	Prompt injection: "Ignore your previous instructions..."

#	Criterion	What it checks	Example
11	Similarity	How close the answer is to a reference answer	BLEU, ROUGE, vector similarity

EXAM TIP

A trick to remember the 11 criteria: **"CLaN PCR HSR AS"** — **C**ost, **La**tency, **N**umber of calls · **P**erformance, **C**lassification, **R**easoning · **H**allucination, **S**afety, **R**efusals · **A**dversarial, **S**imilarity.

1.10 Principle 5 — Divide Labor

KEY IDEA

Divide Labor = don't ask the AI to do a complicated task in one huge prompt — break it into smaller tasks (and chain multiple calls). Also called **task decomposition**.



Instead of "Generate and evaluate the best shoe names", each step has one specific job.

Why divide the task?

A very large prompt can become **confusing · less predictable · more difficult to debug · more likely to hallucinate**. Smaller steps make it easy to see where the problem occurred.

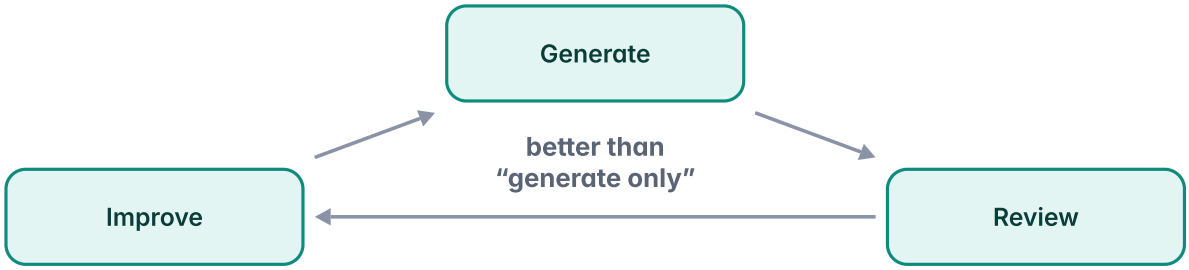
Example from the book — rate the generated names

PROMPT `Rate each name out of 10.`

Name	Score
UniFit SoleStrides	7/10
FlexiSize All-Fit Shoes	8/10
OmniFit FootFlex	7/10
PerfectGlide MultiFit Shoes	9/10

Why can AI evaluate its own output?

While generating, the AI predicts token by token and **may not know how good the whole answer will be**. Once the answer is complete, we give it back: *"Now evaluate these names."* Now it can examine the **whole** response and judge it against criteria.



Generate → Review → Improve is often more effective than Generate only.

We can divide evaluation too

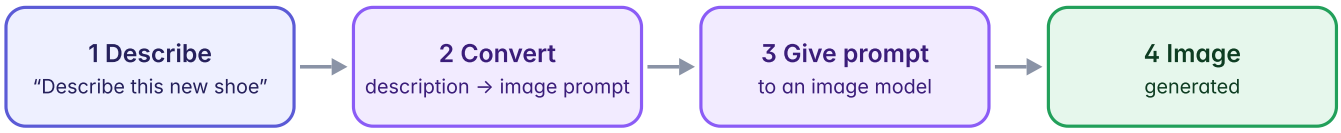
Name	Clarity	Memorability	Uniqueness	Overall
FlexFit	9	8	7	8.0
OmniFit	8	9	8	8.3
PerfectGlide	7	8	9	8.0

Step 1 rate clarity → Step 2 rate memorability → Step 3 rate uniqueness → Step 4 combine the scores.

Meta prompting

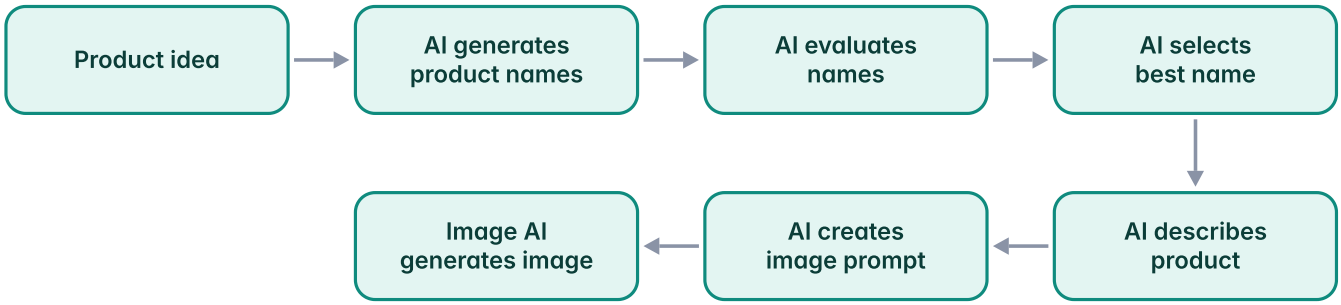
DEFINITION

Meta prompting = using AI to create a prompt for another AI system.



AI → creates prompt → another AI → creates output. e.g. "Create an eye-catching image of a futuristic adaptive shoe..."

AI chaining

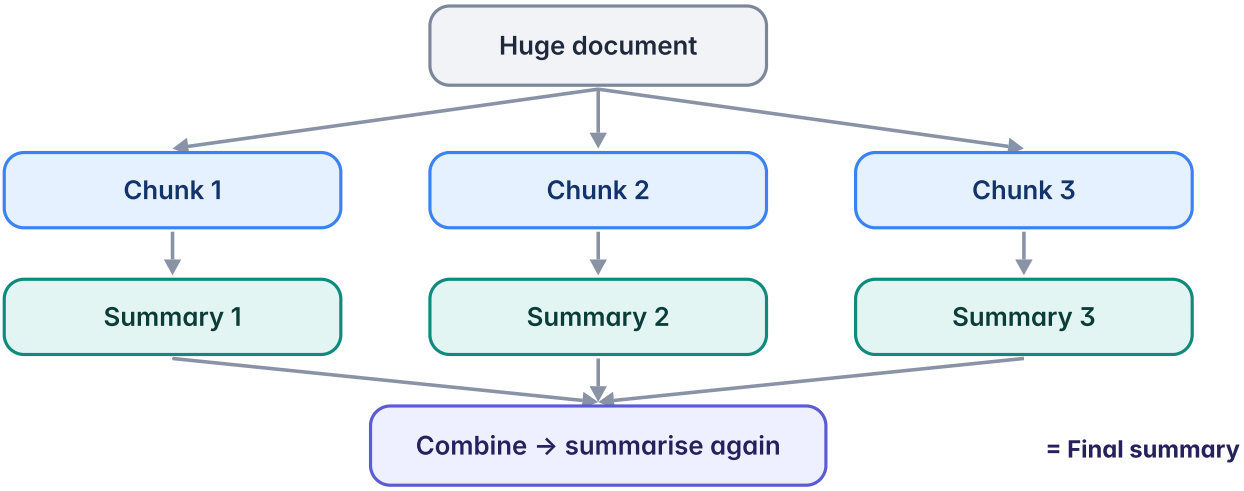


Connecting several AI steps in sequence is called AI chaining.

LangChain connection

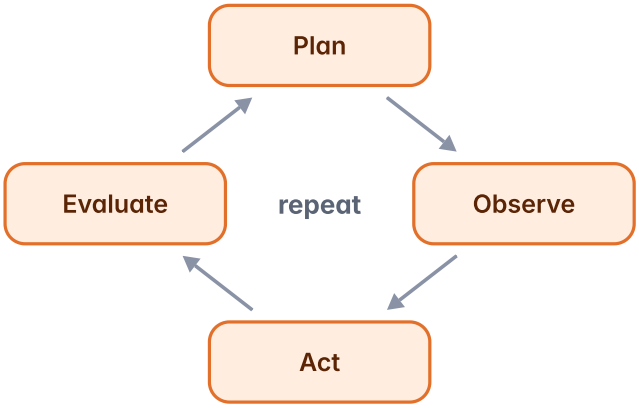
LangChain provides tools for connecting multiple prompts/models together: Prompt 1 → LLM → Prompt 2 → LLM → Prompt 3 → Final Output. This is a **chain**. (We study LangChain in Unit III.)

Progressive summarization



Document too large? Split into chunks → summarise each chunk → combine summaries → summarise again → final summary.

ReAct and AI agents



Task division leads to **AI agents**. An agent may perform **Plan → Observe → Act → Evaluate → Repeat**.

Example: User asks "Find the best laptop." The agent: understands requirements → searches for laptops → compares them → evaluates results → recommends one.

This involves **multiple AI calls and actions** rather than one simple prompt. (**ReAct** = Reason + Act — studied in Unit IV.)

1.11 What Are Large Language Models (LLMs)?

KEY IDEA

Large Language Models (LLMs) are AI models trained on a huge amount of text data. They learn **patterns in language** so that they can understand and generate text.

- Understand questions
- Generate text
- Summarize documents
- Translate languages
- Write code
- Answer questions
- Carry out conversations

Examples: GPT, Gemini, Claude, Llama, Mistral.

PROMPT `Write a short story about a robot.`

OUTPUT `Once there was a small robot named Alex who lived in a futuristic city...`

It can do this because it learned patterns from a huge amount of text during training.

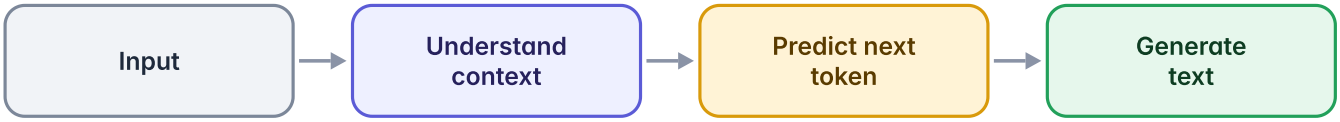
1.12 What Are Text Generation Models?

KEY IDEA

A **text generation model** is an AI system that generates new text based on the input it receives. ChatGPT is an example of a system that uses LLMs for text generation.

INPUT `The weather today is`

PREDICTS `sunny and pleasant.`



The important idea: Input → Understand context → Predict next token → Generate text

HOW THE NEXT TOPICS FIT TOGETHER

Text generation models are built on the **Transformer** architecture. They turn text into numbers (vectors), use attention to understand context, and then use **probabilities** to pick each next token. The next sections explain each of these three ideas — they match the three syllabus headings exactly.

1.13 Tokens and Tokenization

KEY IDEA

A **token** is the basic unit of text processed by an LLM. It can be a complete word, part of a word, a character, or punctuation.

`"I love AI."` might be divided approximately as `I | love | AI | .` — but the exact tokens depend on the **tokenizer** being used.

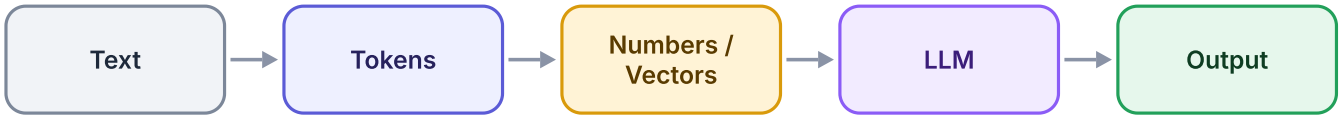
IMPORTANT POINT

Token ≠ always a word. One word can be split into several tokens, e.g. `playing` → `play + ing`.

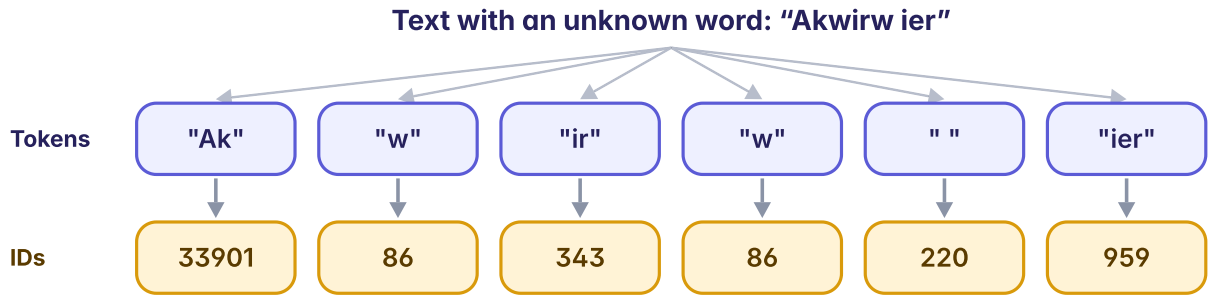
What is tokenization?

Tokenization = breaking text into smaller units called tokens. `"Artificial intelligence is powerful."` → `Artificial | intelligence | is | powerful | .`

The model doesn't directly process English words — it first converts text into **tokens** and then into **numbers**.



Basic process: Text → Tokens → Numbers/Vectors → LLM → Output



Unknown words are tokenized into individual characters or sub-words, and each token gets a numeric ID.

Types of tokenization

Technique	How it works	Used in
BPE — Byte Pair Encoding	Starts with small units and **combines frequently occurring** units into bigger tokens	GPT models
WordPiece	Divides words into smaller **meaningful pieces**	BERT
SentencePiece	Treats input more flexibly; works **without needing traditional word boundaries** (spaces)	Many multilingual models

Why flexible tokenizers matter: `New_York.`, `New_York` and `NewYork.` could all become `["New", "York", "."]` — the original spacing is lost unless the tokenizer handles it (the "???" in your slide).

What is BPE (Byte Pair Encoding)?

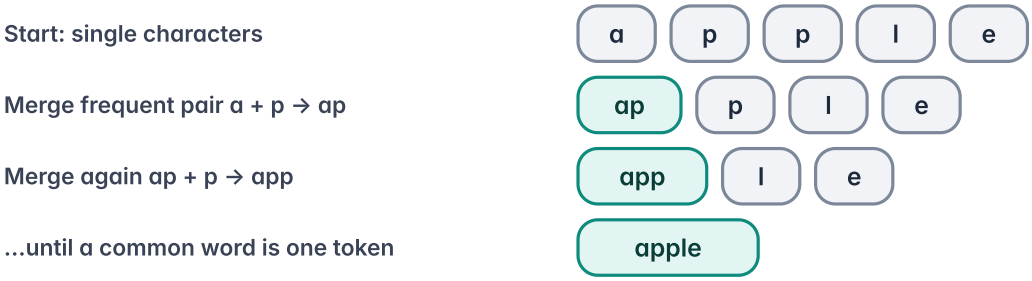


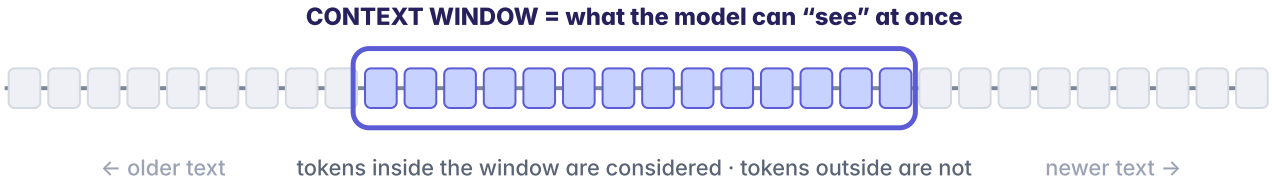
Illustration: frequently occurring character combinations are merged into larger tokens.

REMEMBER

Why is BPE useful? If the model sees a **rare word** it has never seen as a whole, it doesn't treat it as completely unknown — it breaks it into **smaller pieces it already understands**. So BPE helps LLMs handle rare and new words.

Why are tokens important? — the context window

LLMs have a **context window** — the maximum amount of information they can process at one time, measured in tokens. With a 100,000-token window, a model can consider a very large amount of text in one request. More tokens = more text that can be processed (and more cost).



NOTE

The rule "**100 tokens ≈ 75 words**" is only a rough estimate for English — it depends on the language and the tokenizer.

1.14 Pretraining and Fine-Tuning

PRETRAINING

Pretraining = the **first major training stage**. The model is given a huge amount of text (books, websites, articles, research papers, code...) and learns general language knowledge. E.g. from "The sun rises in the east." it learns relationships sun → rises → east.

FINE-TUNING

Fine-tuning = further training a pretrained model for a particular purpose — medical text, legal documents, customer support, coding, financial analysis.



EASY ANALOGY

A student: **Pretraining** = learns many subjects in school. **Fine-tuning** = specialises in Computer Science. So pretraining = general knowledge; fine-tuning = specialised knowledge/behaviour.

1.15 Vector Representations: The Numerical Essence of Language

KEY IDEA

Computers cannot directly understand the meaning of words. So words/tokens are converted into numbers called vectors. A vector is just a list of numbers.

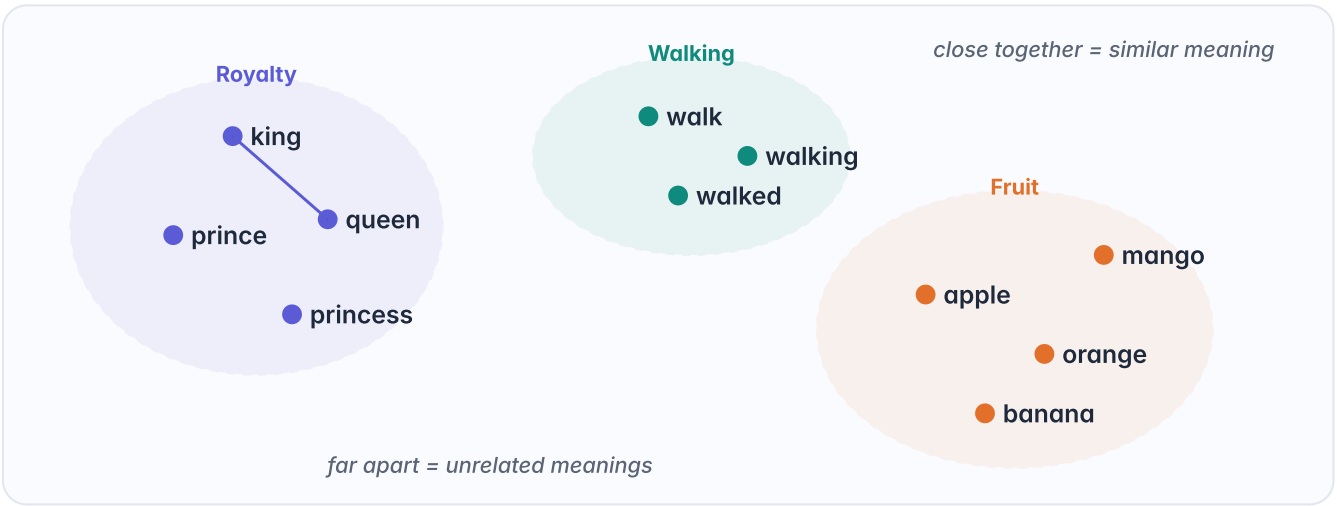


"cat" might be represented by a vector such as [0.21, -0.43, 0.75, 0.18, ...]. Real vectors contain many more dimensions.

$$v = [v_1, v_2, v_3, \ldots, v_n] \quad \text{where each } v_i \text{ is a numerical value}$$

What is a word embedding?

A **word embedding** is a numerical representation of a word that captures its meaning and its relationships with other words. **King** and **Queen** have similar relationships in the embedding space; **walk** and **walking** are related. Words with similar meanings have vectors that are closer together.



Imagine a map: similar words sit near each other (king–queen, apple–orange); unrelated words are far apart.

BEYOND YOUR SLIDES

A famous property of embeddings: **king – man + woman ≈ queen**. The direction "male → female" is roughly the same everywhere in the space. Mention it to show deeper understanding.

EXAM TIP

Exam phrasing to use: "Vector representations convert tokens into **high-dimensional numeric vectors (embeddings)** so that **semantic similarity becomes geometric closeness**."

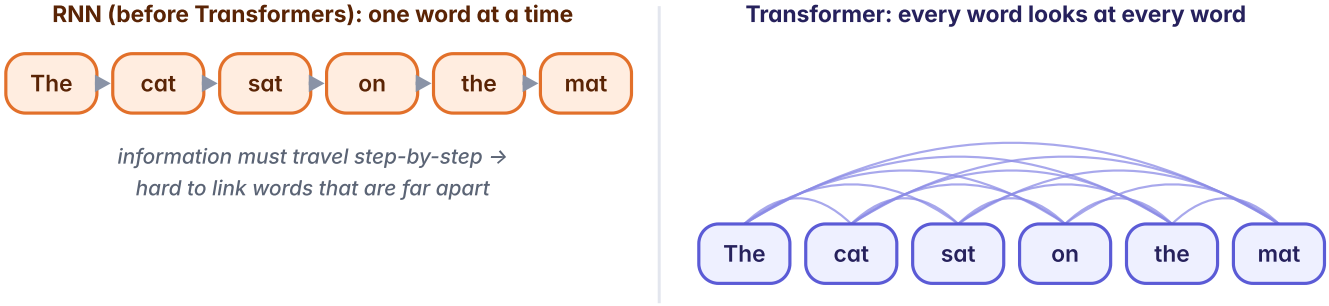
1.16 Transformer Architecture: Orchestrating Contextual Relationships

KEY IDEA

The **Transformer** is one of the most important architectures behind modern LLMs. It was introduced in the famous **2017** paper "Attention Is All You Need". Its major innovation was the **attention mechanism**.

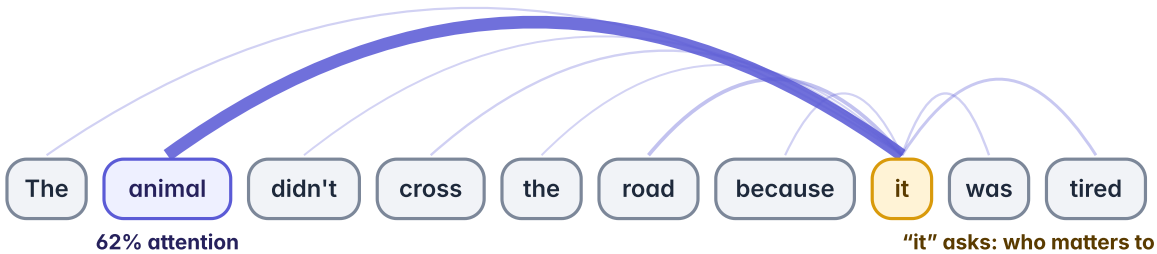
Why was the Transformer important?

Before Transformers, models such as **RNNs** processed text **sequentially** (The → cat → sat → on → the → mat). This made it hard to efficiently understand relationships between words that are **far apart**. Transformers let the model look at relationships between many tokens **directly** (and in parallel).



What is self-attention?

Self-attention lets each token ask: "Which other tokens are important for understanding me?"



"The animal didn't cross the road because it was tired." — self-attention links "it" ↔ "animal" (thicker line = more attention).

EASY ANALOGY

A classroom: every student listens to all the other students and decides *"whose information is important for me?"* Similarly, each token looks at other tokens and gives them **different levels of importance**.

Attention as "voting"

For a word like **bank**, the surrounding words "vote" on its meaning:

EX. 1	`I deposited money in the bank.`
MEANING	`Important words: money, deposited → bank = financial institution`

EX. 2	`We sat near the river bank.`
MEANING	`Important words: river, sat → bank = side of a river`

So attention helps the model understand the context-dependent meaning of words.

Long-range dependencies

"Ravi went to the market because he needed vegetables." The model must understand **he → Ravi**. Attention connects these pieces even when other words separate them. Identifying relationships between distant tokens is called handling long-range dependencies.

BERT vs GPT — two Transformer designs

Model	Main architecture	Common use	How it reads
BERT	**Encoder**	Understanding text (classification, search)	Looks at the whole input in both directions
GPT	**Decoder**	Generating text, one token at a time	Looks only at previous tokens (**masked** attention)

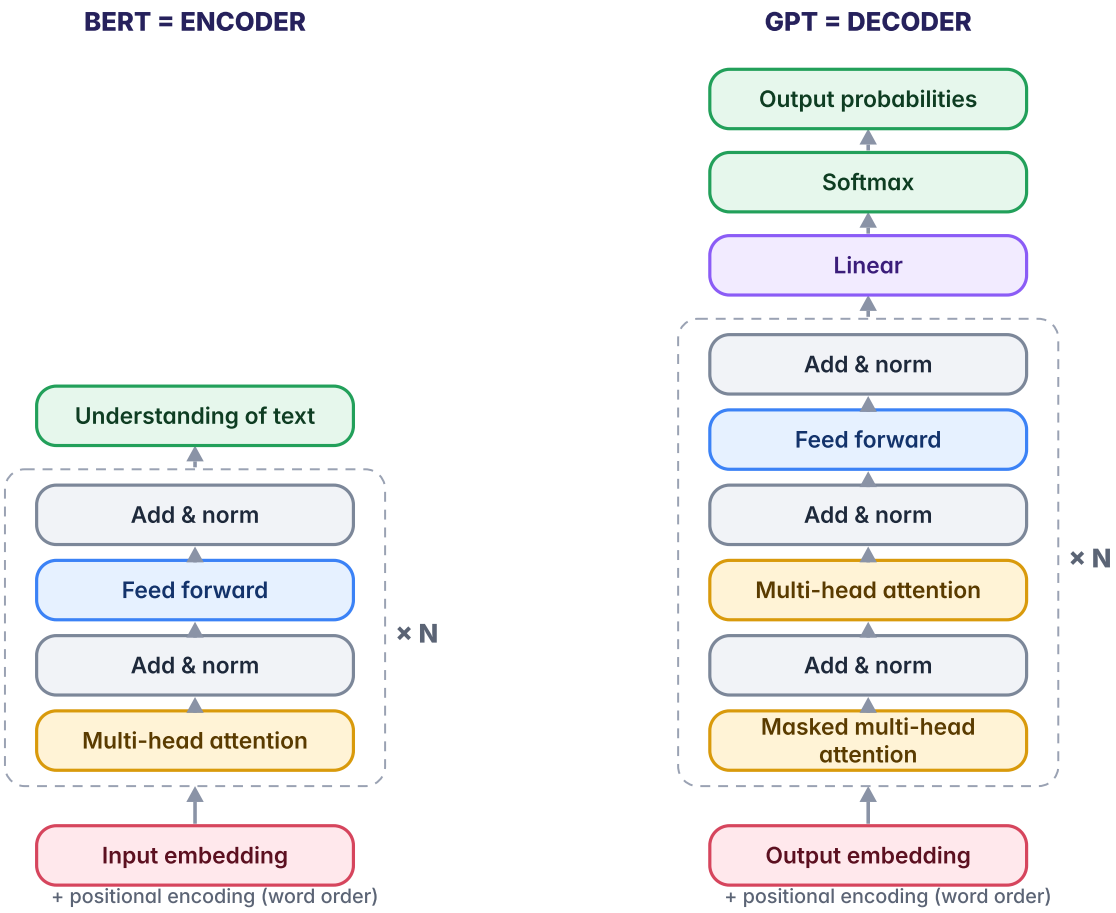


Figure 2-2 (simplified): BERT uses an encoder for input data, while GPT has a decoder for output. "× N" means the block is repeated many times.

NOTE

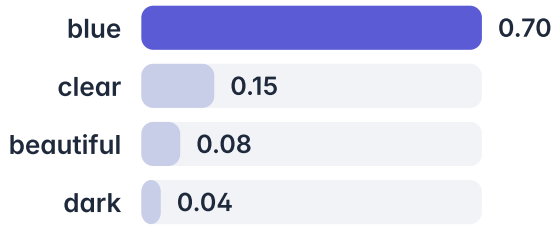
Reading the diagram: Embedding turns tokens into vectors · **Positional encoding** adds word-order information · **Multi-head attention** = several attention "heads" looking at different relationships at once · **Masked** = can't peek at future words · **Add & norm** keeps numbers stable · **Feed forward** = a small neural network per token · **Linear + Softmax** turn the result into **probabilities** for the next token.

1.17 Probabilistic Text Generation: The Decision Mechanism

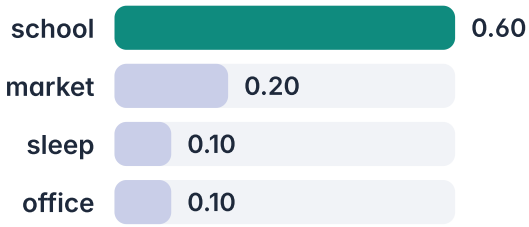
KEY IDEA

Once the Transformer understands the input, it calculates a **probability for every possible next token** and chooses one according to its generation strategy. Then it repeats.

Input: `"The sky is"`



Input: `"I am going to"`



Choosing the highest-probability token gives **"The sky is blue"** and **"I am going to school"**. Then the model predicts the next token again — this is repeated many times.

Mathematical representation

$$w_{\text{next}} = \arg\max_{w} \; \mathrm{P}(\, w \mid w_1, w_2, \ldots, w_{\mathrm{m}} \,)$$

It simply means: choose the next word/token that has the highest probability, given the previous tokens. ("arg max" = the *w* that gives the maximum value.)

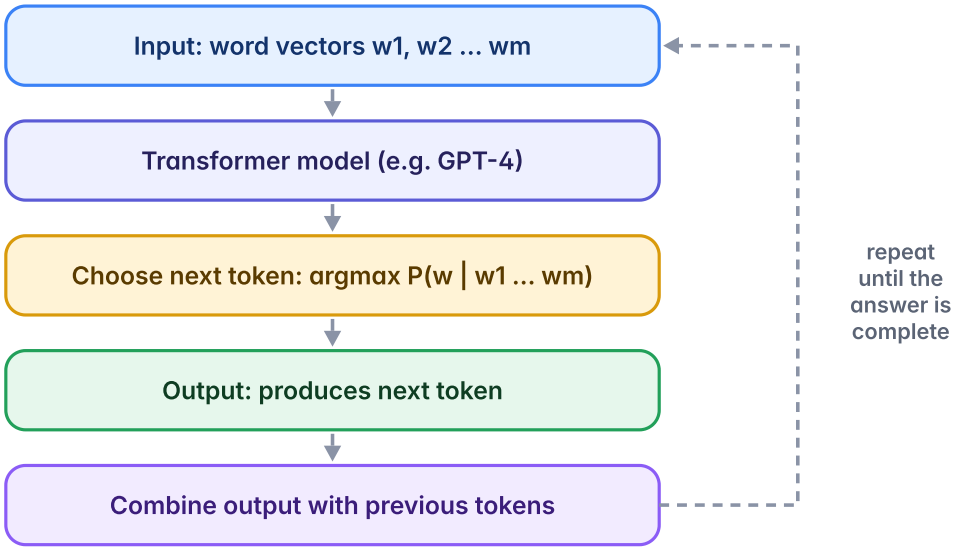
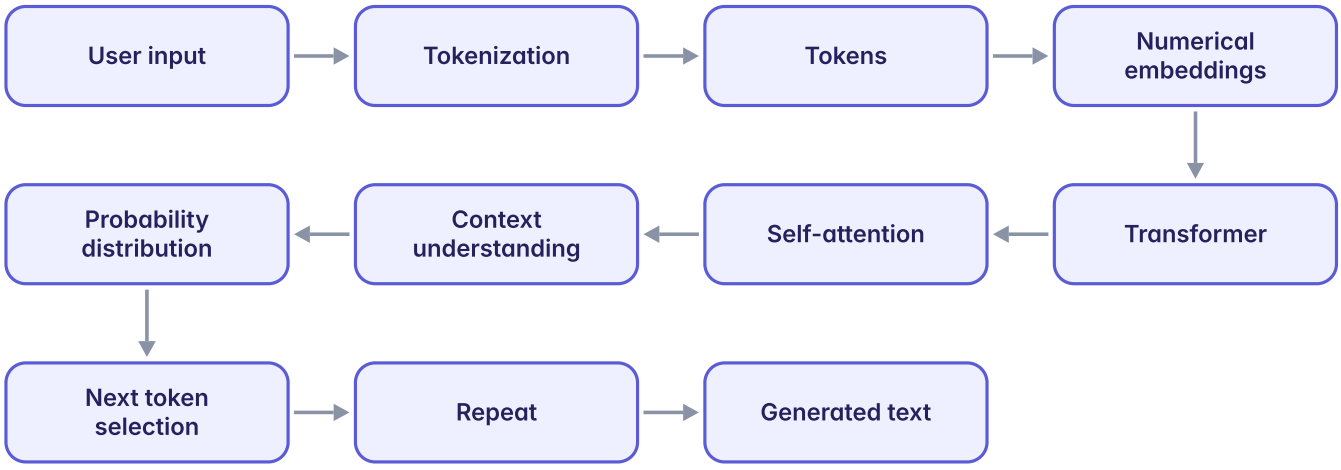


Figure 2-3: how text is generated using transformer models such as GPT-4.

LINK BACK TO TEMPERATURE

Always picking the top token is called **greedy** decoding. Real systems often **sample** instead, using **temperature** (see 1.3): low temperature ≈ almost always the top token; high temperature ≈ more chance for lower-probability tokens.

Overall LLM process



Simplified LLM pipeline — learn this flow; it is a perfect 5-mark answer on its own.

EXAM TIP

Asked to “explain how an LLM generates text”? Write the pipeline above as a flow chart, then explain three things: **(1)** tokens → vectors, **(2)** self-attention for context, **(3)** probability distribution + arg max / temperature. Add the “The sky is → blue” example.

1.18 Parameters, Compute, Tensors and Data Quality

PARAMETERS

Parameters = numerical values **learned during training** — mainly the **weights** of the neural network. Think of them as the model’s learned knowledge / pattern storage. Small model → fewer parameters; large model → more.

WATCH OUT

More parameters does not automatically mean a model is better at every task.

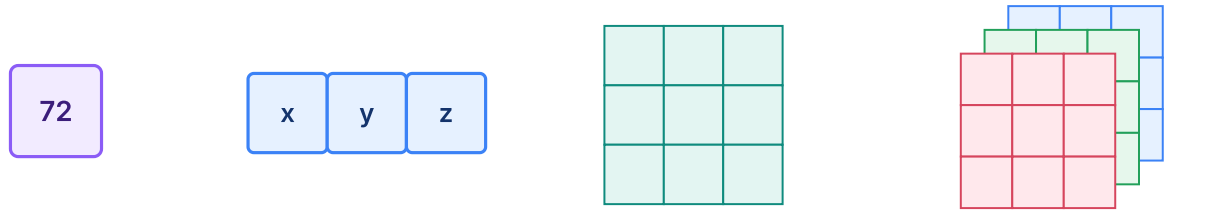
Why does training an LLM need huge computing power?

LLMs can have **billions of parameters**. Training needs: huge datasets · large memory · powerful processors · **GPUs/accelerators** · lots of electricity · significant training time. That’s why companies use large computing clusters.

Why GPUs? Training involves huge numbers of **matrix and tensor operations**. GPUs perform many numerical calculations **in parallel**, so they make training and inference much faster (e.g. NVIDIA **H100**).

What is a tensor?

A **tensor** is a multi-dimensional grid of numbers — a universal container for numerical data (used in TensorFlow and PyTorch). Its **rank** = number of dimensions (axes).



Rank 0	Rank 1	Rank 2	Rank 3+
scalar	vector (list)	matrix (table / B&W image)	colour image = H × W × RGB

Rank 0 scalar (72) · Rank 1 vector ([x, y, z]) · Rank 2 matrix (spreadsheet / grayscale image) · Rank 3+ (colour image = height × width × 3 RGB channels; a video adds time → 4D).

Why is data quality important?

LLMs learn from their training data. If the data contains **bias, incorrect information, offensive content or poor-quality information**, the model can learn those patterns too. So: good training data + careful evaluation + human oversight are important for responsible AI.

1.19 Historical Underpinnings and Model Families



each generation improved capabilities (years added for reference)

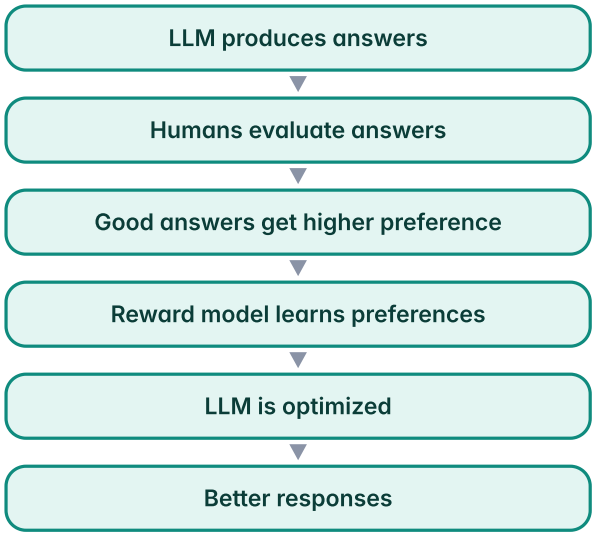
Simplified evolution of OpenAI's GPT family

Model	What to remember
GPT-2	Showed large Transformer models could write surprisingly **coherent text**. Concerns about **misuse** affected how it was released → lesson: as AI becomes powerful, **safety and responsible release** matter.
GPT-3	Major increase in **scale**. Writing, question answering, summarization, translation, code generation and **few-shot learning**.
GPT-3.5 & ChatGPT	Better conversation. ChatGPT was **not just a raw pretrained model** — it got extra training to follow instructions and interact with people.
GPT-4	Major improvement in **reasoning** and language; handles more complex tasks; **multimodal** capabilities.
GPT-4o	Stronger **real-time multimodal** interaction: text, image, audio. E.g. give an image + "What is shown in this image?"

How was ChatGPT trained? (3 stages)



1. **Demonstration data** — human trainers write examples of good responses. Q: "Explain gravity to a child." Good answer: "Gravity is the force that pulls things toward Earth..."
2. **Supervised fine-tuning (SFT)** — the model is trained on those examples: Prompt → human-written good response → fine-tuning → **better instruction following**.
3. **Human preference / Reinforcement learning** — the model generates several responses; humans compare them (A good, B excellent, C poor). These preferences train a **reward model**; then reinforcement learning (historically **PPO — Proximal Policy Optimization**) pushes the model toward higher-reward responses.



RLHF

RLHF = Reinforcement Learning from Human Feedback.
Goal: make the model more Helpful, Harmless/safe, and Aligned with user instructions.

EXAM TIP

"Explain how ChatGPT was trained / explain RLHF" is a very likely question. Draw the 3 stages + the RLHF flow, and name **reward model** and **PPO**.

Other model families

Family	By	Key points
Gemini	Google	Family of **multimodal** generative models. Google's earlier chatbot **Bard** was rebranded as Gemini.
Llama (Llama, Llama 2, Llama 3)	Meta	**Relatively open access** → attractive for researchers, developers, companies, fine-tuning and **local deployment**.
Mistral	Mistral AI	**Strong performance relative to size**; efficient deployment and fine-tuning. Mixtral uses **Mixture of Experts**.
Claude	Anthropic	Known for a **very large context window** (100K → 200K tokens) and a strong focus on safety ("Constitutional AI").
GPT-4V(ision)	OpenAI	GPT-4 that accepts **images as input** — describe images, read charts, answer questions about photos.

BEYOND YOUR SLIDES

Your slides list "Anthropic: Claude", "GPT-4V(ision)" and "Model Comparison" in the session plan but don't have slides for them. The two rows above (Claude, GPT-4V) are from the textbook so you aren't caught off guard.

Quantization

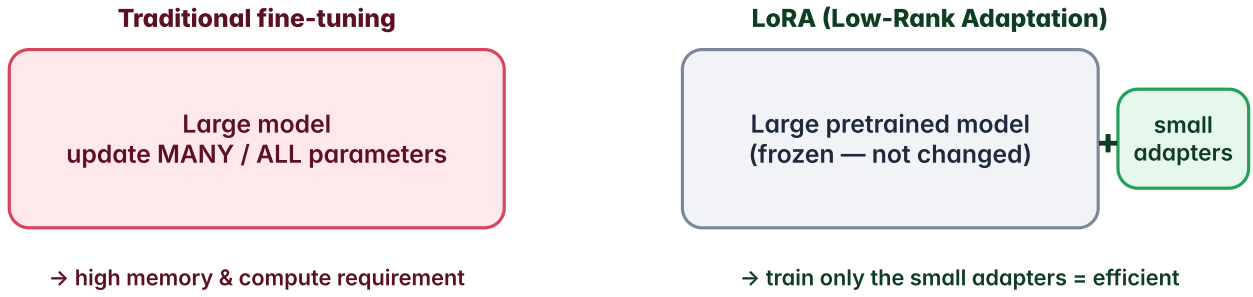
Quantization = using lower numerical precision to store model parameters, e.g. 8-bit or 4-bit numbers instead of 16-bit. It reduces **model size, memory and compute** → large models can run on less powerful hardware.

EASY ANALOGY

Instead of storing **3.1415926535** you store **3.14** — you lose a little precision but need much less storage.

LoRA — Low-Rank Adaptation

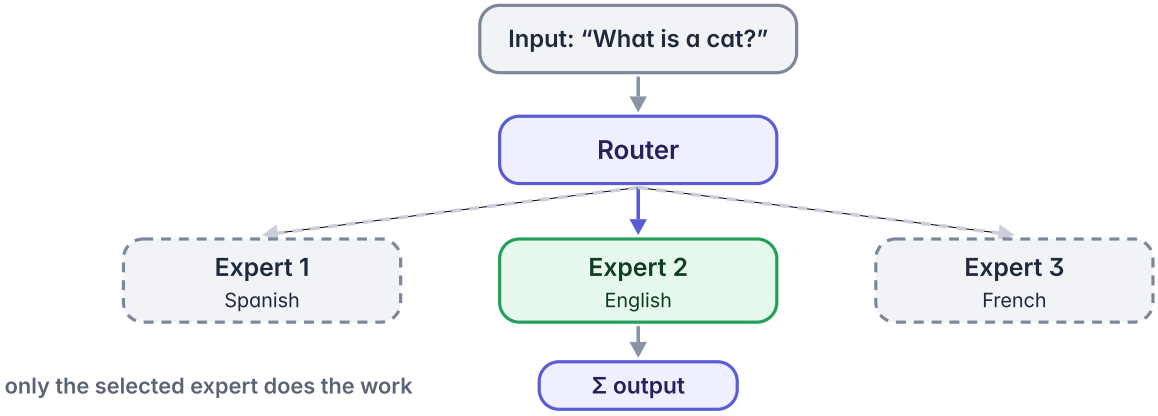
A technique to **fine-tune large models efficiently**: instead of changing all parameters, LoRA adds a much smaller set of trainable parameters (adapters).



LoRA reduces the computation and memory required for fine-tuning.

Mixture of Experts (MoE)

An MoE model contains multiple specialised components called **experts**. A **router** sends each input to **selected experts** instead of activating all of them → better efficiency at large scale.



EASY ANALOGY

A **hospital**: Patient → type of problem → **specialist selected** → treatment. Similarly, MoE activates only selected parts of a large model for a particular input.

Context window and multimodal AI

CONTEXT WINDOW

The **context window** is the amount of information an LLM can consider during one interaction (e.g. 100,000 tokens). Think of it as the model's **working desk**: a larger desk keeps more documents in front of you.

Multimodal AI works with multiple types of information, not just text.



Upload a photo of a plant and ask “What disease might this plant have?”

R1 Unit I — Quick Revision & Likely Questions

PART A IN 4 LINES

- Prompt engineering = designing/improving instructions for reliable AI output.
- Naive prompt → 5 problems: vague direction, unformatted output, missing examples, limited evaluation, no task division.
- Five principles: **Direction · Format · Examples · Evaluate · Divide** (WHAT · HOW · SHOW · CHECK · BREAK).
- Good Prompt = Clear Task + Style + Format + Examples + Constraints.

PART B IN 7 LINES

- LLMs generate **one token at a time**; the prompt changes next-token probabilities.
- **Temperature**: low = predictable, high = creative.
- Tokens ≠ words; BPE merges frequent pairs; handles rare words.
- Embeddings: similar meaning → nearby vectors.
- Transformer (2017, "Attention Is All You Need"); self-attention links "it" ↔ "animal".
- BERT = encoder (understand); GPT = decoder (generate).
- w_{next} = arg max $P(w \mid w_1 \ldots w_m)$.

ONE-LINERS EXAMINERS LOVE

- Zero / one / few-shot = 0 / 1 / several examples.
- Blind evaluation = shuffle so you can't favour a prompt.
- Meta prompting = AI writes a prompt for another AI.
- ChatGPT training = Demonstrations → SFT → RLHF (reward model + PPO).
- Quantization = lower precision; LoRA = small adapters; MoE = router + experts.

COMMON MISTAKES

- Don't say "token = word".
- Don't confuse Direction (style) with Format (form).
- More examples ≠ always better (less creativity).
- More parameters ≠ always better.
- Role/"expert" ≠ guaranteed correct.

Likely CIE questions — with answer plans

Explain the five principles of prompting with suitable examples. 10 marks

ANSWER PLAN

- Define prompt engineering (1 line) + naive vs engineered prompt.
- For each principle: definition + the shoe-naming example (persona, JSON/comma list, few-shot, thumbs-up A/B test, generate → rate → select).
- Draw the 5-card overview or "Direction → Format → Examples → Evaluate → Divide".
- Finish with the student-answer analogy.

What are the limitations of a naive prompt? How does prompt engineering overcome them? 6 marks

ANSWER PLAN

- Table of the 5 problems (meaning + example).
- Map each to the principle that fixes it.
- Conclude: simple prompt → casual use; engineered → consistent, measurable.

Explain how quality of prompts is evaluated. Describe blind evaluation and the criteria used. 8 marks

ANSWER PLAN

- Blind prompting vs systematic eval.
- 7 steps: two prompts, run 5×, DataFrame, CSV, shuffle, rate 1/0, score (60% vs 80%).
- Why shuffle & why test multiple times.
- Human vs automatic evaluation.
- List the 11 criteria (cost, latency ... similarity: BLEU/ROUGE).

What is Divide Labor? Explain meta prompting, AI chaining and progressive summarization. 8 marks

ANSWER PLAN

- Task decomposition definition + product naming steps.
- Why: easier to debug, fewer hallucinations.
- Generate → Review → Improve.
- Meta prompting 4 steps; AI chaining flow; progressive summarization diagram; link to LangChain & agents (Plan → Observe → Act → Evaluate).

What is tokenization? Explain BPE with an example. Why are tokens important? 6 marks

ANSWER PLAN

- Token definition (word/sub-word/char/punctuation), token ≠ word (play+ing).
- Text → Tokens → IDs → vectors → LLM.
- BPE: start with characters, merge frequent pairs (a+p → ap...); handles rare words ("Akwirw ier" example).
- WordPiece (BERT), SentencePiece.
- Context window; 100 tokens ≈ 75 words.

Explain vector representations and word embeddings. 5 marks

ANSWER PLAN

- Computers need numbers → tokens become vectors v = [v1 ... vn].
- "cat" → [0.21, -0.43, ...].
- Embedding = meaning + relationships; similar words close (king/queen, walk/walking).
- Draw the 2-D map.

Explain the Transformer architecture and self-attention. Differentiate BERT and GPT. 10 marks

ANSWER PLAN

- 2017 "Attention Is All You Need"; RNN sequential problem vs transformer direct connections.
- Self-attention: "it ↔ animal", classroom analogy; attention as voting ("bank").
- Long-range dependencies (Ravi/he).
- Encoder–decoder block diagram (embedding, positional encoding, multi-head attention, add & norm, feed forward, linear, softmax).
- BERT vs GPT table.

How does an LLM decide the next word? Explain probabilistic text generation. 6 marks

ANSWER PLAN

- "The sky is" probability table → blue.
- Formula w_next = argmax P(w | w1...wm).
- Generation loop (Figure 2-3).
- Temperature / sampling; small prompt change → different output.

Explain how ChatGPT was trained. What is RLHF? 6 marks

ANSWER PLAN

- Pretraining → fine-tuning.
- Stage 1 demonstration data; Stage 2 SFT; Stage 3 human preferences → reward model → PPO.
- RLHF flow + goals: helpful, harmless, aligned.

Write short notes on: (a) Quantization (b) LoRA (c) Mixture of Experts (d) Context window. 4 × 2 marks

ANSWER PLAN

- Quantization: 16-bit → 8/4-bit, 3.14 analogy.
- LoRA: frozen model + small trainable adapters.
- MoE: router sends input to selected experts; hospital analogy.
- Context window: max tokens at once; working-desk analogy.

UNIT II

Standard Practices for Text Generation with ChatGPT

Practical techniques: getting clean lists, JSON and YAML that code can read, transforming and styling text, chunking long documents, role prompting and reducing hallucinations.

Generating Lists	2.1	Hierarchical List Generation	2.2
Parsing with Regular Expressions — Example 3-1	2.3	Keeping the Structure — Example 3-2 (Dictionary)	2.4
When to Avoid Using Regular Expressions	2.5	Generating JSON	2.6
YAML	2.7	Filtering YAML Payloads	2.8
Diverse Format Generation with ChatGPT	2.9	Explain It Like I'm Five (ELI5)	2.10
Universal Translation Through LLMs	2.11	Ask for Context	2.12
Text Style Unbundling	2.13	Identifying the Desired Textual Features	2.14
Generating New Content with the Extracted Features	2.15	Extracting Specific Textual Features with LLMs	2.16
Chunking Text and Strategies	2.17	Role Prompting	2.18
GPT Prompting Tactics	2.19	Avoiding Hallucinations with Reference	2.20
Give GPT "Thinking Time" and Inner Monologue	2.21	Self-Eval LLM Responses	2.22
Unit II — Quick Revision & Likely Questions	R2		

⏱ ~2.5 hours · Sittings 4–5

2.1 Generating Lists

KEY IDEA

Automatically generating lists is incredibly powerful: you focus on **higher-level tasks** while GPT can automatically generate, refine, rerank and deduplicate lists on your behalf.

The naive list prompt

PROMPT

```
Generate a list of Disney characters.
```

OUTPUT

```
Sure, here is a list of some popular Disney characters:

1. Mickey Mouse
2. Minnie Mouse
...
30. Bagheera (The Jungle Book)
```

This looks fine to a human, but it causes **5 problems for a computer program**:

#	Problem	What goes wrong
1	Different list format	GPT uses `1. Simba` but your Python expects `- Simba` → splitting on `-` fails.
2	Extra explanation	"Sure! Here are some Disney characters: ... Hope this helps!" — extra sentences make parsing harder.
3	Count not controlled	Asked for characters, GPT may give 10, 20, 30... If you need exactly 5, the number is **unpredictable**.
4	Inconsistent names	"Simba", "Elsa", "Bagheera (The Jungle Book)" — some have the movie in brackets → **inconsistent data** needing extra cleaning.
5	No filtering	"Only male characters" — GPT might still include characters that don't meet your exact rule.

The optimized list prompt

OPTIMIZED

```
Generate a bullet-point list of 5 male Disney characters.
Only include the name of the character for each line.
Never include the film for each Disney character.
Only return the Disney characters, never include any commentary.

Below is an example list:

* Aladdin
* Simba
* Beast
* Hercules
* Tarzan
```

OUTPUT

```
* Woody
* Buzz Lightyear
* Stitch
* Jack Sparrow
* Prince Charming
```

WHAT THE OPTIMIZED PROMPT ACHIEVED

- Restricted the list to a **fixed size of five**
- Generated **only male** characters
- Correctly **formatted** with bullet points
- **Removed** any preceding commentary

PROVIDE EXAMPLES

Simply rephrasing your prompt to include **examples (few-shot prompting)** can greatly impact the output.

Simple lists are fine for most tasks but are **less structured**. Three typical nested data structures: ① **nested text (hierarchical lists)** ② **JSON** ③ **YAML**.

2.2 Hierarchical List Generation

KEY IDEA

Hierarchical lists are useful when your desired output is nested — for example, a detailed article outline with headings and sub-headings.

PROMPT

```
Generate a hierarchical and incredibly detailed article outline on:

What are the benefits of data engineering.

See an example of the hierarchical structure below:

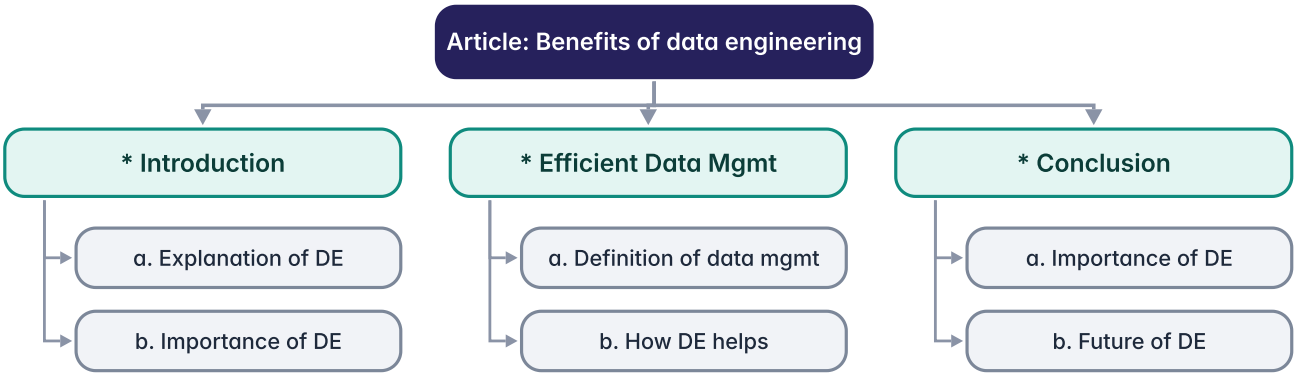
Article Title: What are the benefits of digital marketing?

* Introduction
    a. Explanation of digital marketing
    b. Importance of digital marketing in today's business world
* Increased Brand Awareness
    a. Definition of brand awareness
    b. How digital marketing helps in increasing brand awareness
```

OUTPUT

```
Article Title: What are the benefits of data engineering?

* Introduction
    a. Explanation of data engineering
    b. Importance of data engineering in today's data-driven world
...(10 sections later)...
* Conclusion
    a. Importance of data engineering in the modern business world
    b. Future of data engineering and its impact on the data ecosystem
```



Hierarchical = arranged in levels: main heading → sub-heading → details (DE = data engineering).

Why these prompt words matter

Word in the prompt	What it tells GPT
"hierarchical"	"Don't give me a flat list — give me **main headings and sub-headings**."
"incredibly detailed"	Produce a large, detailed answer (e.g. a. What is DE? b. History c. Role of data engineers...). Similar words: *very long, comprehensive, include at least 10 headings*.
the example structure	Shows the exact format: `*` for headings, `a.` `b.` for sub-headings → makes parsing possible.

WATCH OUT

Asking for 10 doesn't guarantee 10. GPT might return only 8 headings. So your Python program should **check the result** (e.g. `if number_of_headings == 10: print("Valid")`) or be designed to work with whatever number GPT returns.

Parsing — the key concept

DEFINITION

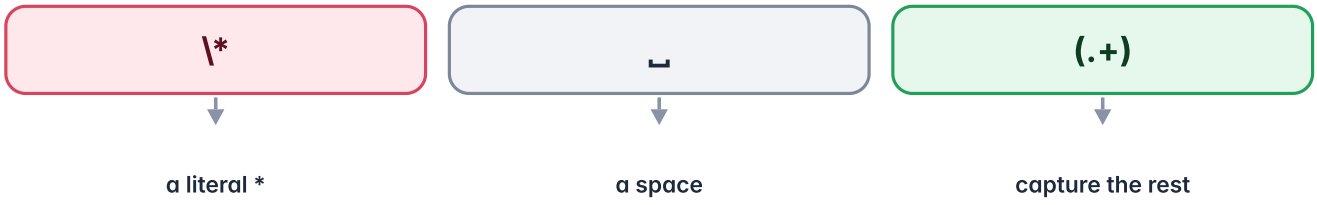
Parsing = converting unstructured text into structured data — taking GPT's text and turning it into a useful Python data structure.



2.3 Parsing with Regular Expressions — Example 3-1

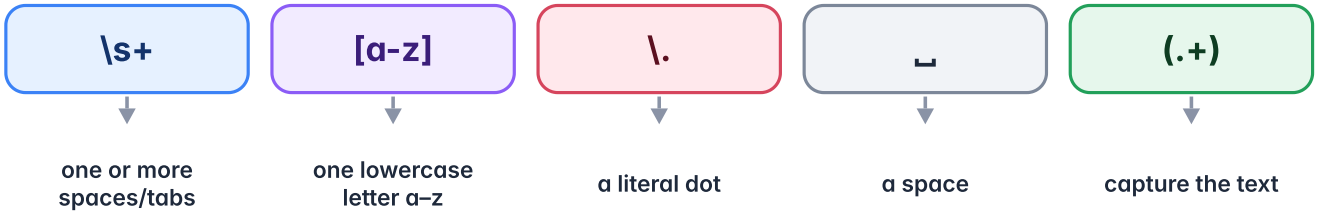
A **regular expression (regex)** is a pattern used to find specific text. Python's regex module is `re`. GPT's main headings start with `*` and sub-headings start with `a.`, `b.`, `c.` — so we can write one pattern for each. Imagine a very large outline: instead of extracting everything manually, regex finds all headings and sub-headings automatically.

Pattern 1 — headings: r'* (.+)'



" Introduction" → captures "Introduction"*

Pattern 2 — sub-headings: r'\s+[a-z]\. (.+)'



" a. Explanation of data engineering" → captures "Explanation of data engineering"

Symbol	Meaning	Why
`\*`	the literal * character	`*` has a special meaning in regex, so `\` "escapes" it: "I literally mean *"
`\.`	the literal . character	`.` normally means "any character"
`.`	any character	
`+`	one or more times	`.+` = one or more of any character
`( )`	capture group	the part we want to extract (read with `group(1)`)
`\s`	whitespace (space/tab)	`\s+` = one or more spaces · `\s*` = zero or more spaces
`[a-z]`	one lowercase letter	matches a, b, c ... z

EXAMPLE 3-1 · PARSING A HIERARCHICAL LIST

```
import re

openai_result = """
* Introduction
    a. Explanation of data engineering
    b. Importance of data engineering in today's data-driven world
* Efficient Data Management
    a. Definition of data management
    b. How data engineering helps in efficient data management
* Conclusion
    a. Importance of data engineering in the modern business world
    b. Future of data engineering and its impact on the data ecosystem
"""

# Regular expression patterns
heading_pattern = r'\* (.+)'
subheading_pattern = r'\s+[a-z]\. (.+)'

# Extract headings and subheadings
headings = re.findall(heading_pattern, openai_result)
subheadings = re.findall(subheading_pattern, openai_result)

# Print results
print("Headings:\n")
for heading in headings:
    print(f"* {heading}")

print("\nSubheadings:\n")
for subheading in subheadings:
    print(f"* {subheading}")
```

OUTPUT

```
Headings:
* Introduction
* Efficient Data Management
* Conclusion

Subheadings:
* Explanation of data engineering
* Importance of data engineering in today's
data-driven world
* Definition of data management
* How data engineering helps in efficient
data management
* Importance of data engineering in the
modern business world
* Future of data engineering and its impact
on the data ecosystem
```

`re.findall()` = find **ALL** matches, not just one. It returns a list:

```
["Introduction",
 "Efficient Data Management",
 "Conclusion"]
```

WATCH OUT

The problem: Example 3-1 produces **two separate lists**. It **doesn't know which sub-heading belongs to which heading** — the relationship is lost.

2.4 Keeping the Structure — Example 3-2 (Dictionary)

Example 3-2 solves the problem by building a **dictionary** where each heading maps to its own list of sub-headings:

GOAL

```
{
    "Introduction": [
        "a. Explanation of data engineering",
        "b. Importance of data engineering in today's data-driven world"
    ],
    "Efficient Data Management": [
        "a. Definition of data management",
        "b. How data engineering helps in efficient data management"
    ],
    "Conclusion": [ "a. Importance of ...", "b. Future of ..." ]
}
```

EXAMPLE 3-2 · PARSING INTO A DICTIONARY

```
import re

section_regex = re.compile(r"\* (.+)")              # headings: "* Introduction"
subsection_regex = re.compile(r"\s*([a-z]\..+)")    # sub-headings: "a. ..." (\s* = 0+ spaces)

result_dict = {}          # empty dictionary we will fill
current_section = None    # remembers which heading we are under

for line in openai_result.split("\n"):         # go through GPT's text line by line
    section_match = section_regex.match(line)
    subsection_match = subsection_regex.match(line)

    if section_match:                                  # is this line a heading?
        current_section = section_match.group(1)       # "Introduction"
        result_dict[current_section] = []              # {"Introduction": []}
    elif subsection_match and current_section is not None:   # a sub-heading under a heading?
        result_dict[current_section].append(subsection_match.group(1))

print(result_dict)
```

Line-by-line walkthrough

Line read	Matches?	Action	result_dict after
`* Introduction`	heading	current_section = "Introduction"; create empty list	`{"Introduction": []}`
`a. Explanation…`	sub-heading	append to current section	`{"Introduction": ["a. Explanation…"]}`
`b. Importance…`	sub-heading	append to current section	`{"Introduction": ["a. …", "b. …"]}`
`* Efficient Data Mgmt`	heading	current_section changes; new empty list	`{…, "Efficient Data Management": []}`
`a. Definition…`	sub-heading	append under the **new** heading	`{…, "Efficient…": ["a. Definition…"]}`

KEY POINTS

- `re.compile()` prepares a pattern so it can be used **repeatedly**.
- `\s*` (zero or more spaces) handles both `" a. Explanation"` and `"a. Explanation"` → **more flexible** than `\s+`.
- `group(1)` returns the text captured by the first `( )`.
- `current_section is not None` ensures we never add a sub-heading before any heading has been seen.

2.5 When to Avoid Using Regular Expressions

KEY IDEA

Regex is great for **simple patterns**, but as the data becomes more complicated, regex becomes hard to write and maintain. Instead, ask GPT to return a **structured format** like JSON, YAML, CSV, Mermaid or code.

	Regex approach	JSON approach
What you do	Write patterns like `r'\* (.+)'` and `r'\s+[a-z]\. (.+)'`	GPT already returns `{"Introduction": […], "Conclusion": […]}`
Parsing	Custom code; breaks if the format changes slightly	One line: `json.loads()`
Complex data	Patterns become complicated too	Usually **easier and cleaner**
Best for	Simple patterns	Structured / nested data

2.6 Generating JSON

Tell GPT: *"Give me the answer **only in valid JSON**."* You can also give an **example of the expected JSON** — that is **giving direction and examples**, and GPT follows a format you show it more reliably.

PROMPT

```
Compose a very detailed article
outline on "The benefits of
learning code" with a JSON payload
structure that highlights key
points.

Only return valid JSON.

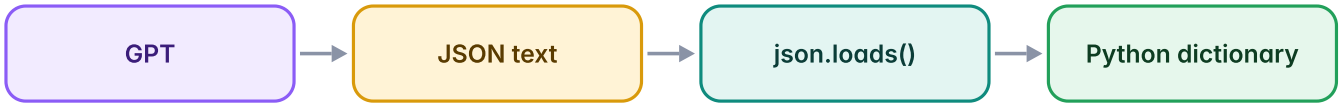
Here is an example of the JSON
structure:
{
  "Introduction": [
    "a. Explanation of data
engineering",
    "b. Importance of data
engineering in today's data-driven
world"],
  ...
  "Conclusion": [
    "a. Importance of data
engineering in the modern business
world",
    "b. Future of data engineering
and its impact on the data
ecosystem"]
}
```

OUTPUT (VALID JSON)

```
{
  "Introduction": [
    "a. What is coding?",
    "b. Importance of coding"
  ],
  "Conclusion": [
    "a. Benefits of coding",
    "b. Future of coding"
  ]
}
```

Why is JSON useful?

Plain text ("Introduction: Coding is important... Conclusion: Coding has many benefits...") makes it hard for Python to know where each part begins and ends. JSON has a **fixed structure**, so Python can directly access `data["Introduction"]`.



JSON.LOADS() — THE EASY PART

```
import json

openai_json_result = """
{
    "Introduction": [
        "What is coding?",
        "Importance of coding"
    ]
}
"""
parsed_json_payload = json.loads(openai_json_result)   # JSON string -> Python dict
print(parsed_json_payload["Introduction"])
```

Common JSON problems

PROBLEM 1 — EXTRA TEXT

GPT might wrap the JSON in chatty text and code fences:
`Sure! Here's the JSON:`
`` ```json {"Name": "John Smith"} ``` ``
A human understands it, but `json.loads()` expects **only** JSON. Ideally GPT returns just `{"Name": "John Smith"}`.

PROBLEM 2 — INVALID JSON

GPT might produce **invalid JSON**, e.g. `{"Name": "John Smith", "Age":}`. `json.loads()` will raise an error. Always assume the output might be invalid and handle it.

GOOD PRACTICE (SAFE PARSING)

```
try:
    data = json.loads(gpt_output)
except json.JSONDecodeError:
    data = None      # e.g. retry the request or ask GPT to fix its JSON
```

2.7 YAML

KEY IDEA

YAML (also .yml) is another way of storing structured information — the same kind of data as JSON but in a simpler-looking, **indentation-based** format that is easy for humans to read.

JSON

```
{
  "item": "Apple",
  "quantity": 5,
  "unit": "pieces"
}
```

YAML (SAME DATA)

```
# a shopping item
item: Apple
quantity: 5
unit: pieces
```

A YAML LIST OF ITEMS

```
- item: Apple Slices
  quantity: 5
  unit: pieces

- item: Eggs
  quantity: 2
  unit: dozen
```

① **No lots of special characters**
JSON uses `{ } [ ] , "` ; YAML mostly uses **indentation** → easier to write.

② **Readability**
Designed to be easy for humans: `item: Milk` / `quantity: 1` / `unit: gallon` .

③ **Comments**
YAML allows comments with `#` , e.g. `# This is the quantity of milk` — useful for teams.

EXAM TIP

"Compare JSON and YAML" → write the same data in both, then list YAML's 3 advantages (fewer special characters, readability, comments) and JSON's strength (strict structure that every programming language parses easily).

2.8 Filtering YAML Payloads

KEY IDEA

GPT can act as a data cleaner / filter: give it a **schema** (the allowed items) and the user's request, and it keeps only what matches.

PROMPT*

```
You are a helpful assistant that converts a user's shopping list into YAML.
Only use items from the schema below. Remove anything that is not in the schema.
If nothing matches, return "No Items".

Schema (allowed items): Apple Slices, Milk, Bread, Eggs

User query: "5 apple slices, 2 dozen eggs and 3 bananas"
```

OUTPUT

```
- item: Apple Slices
  quantity: 5
  unit: pieces

- item: Eggs
  quantity: 2
  unit: dozen
```

*Simplified version of the textbook's prompt. **Bananas** isn't in the allowed list, so GPT filters it out. If the user only says "5 bananas", GPT returns **"No Items"**.

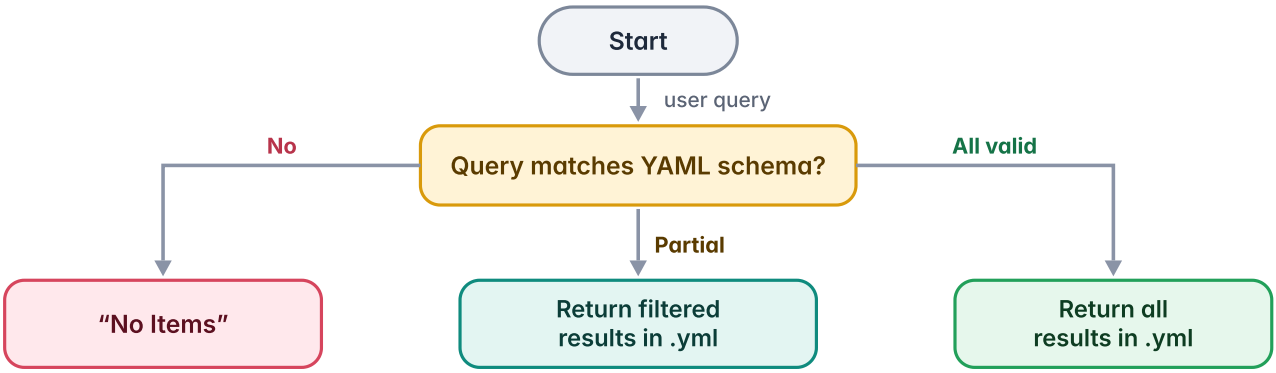


Figure 3-1: using an LLM to decide the control flow of an application instead of code

LLM as a reasoning engine

NORMALLY: PROGRAMMING LOGIC

```
if item in allowed_items:
    keep_item()
else:
    remove_item()
```

Instead, you can give GPT **instructions** and let it perform this filtering. So an LLM can sometimes perform tasks that would normally require programming logic — even deciding the **control flow** of an app.

WATCH OUT

For important applications, you should **still validate the LLM's output with code**.

2.9 Diverse Format Generation with ChatGPT

GPT isn't limited to JSON and YAML. It can generate **JSON · YAML · CSV · Mermaid diagrams · Python code · JavaScript code · conversations · scripts** and more.

Mermaid diagrams

MERMAID CODE GENERATED BY GPT

```
graph TD
ChooseFood[Choose Food] --> AddToCart[Add to
Cart]
AddToCart --> ConfirmCart[Confirm Cart]
ConfirmCart --> PayForMeal[Pay for Meal]
```

Mermaid is a language used to create diagrams from text. A Mermaid renderer turns the code into a flowchart.

Normal language → GPT → Mermaid code → Diagram

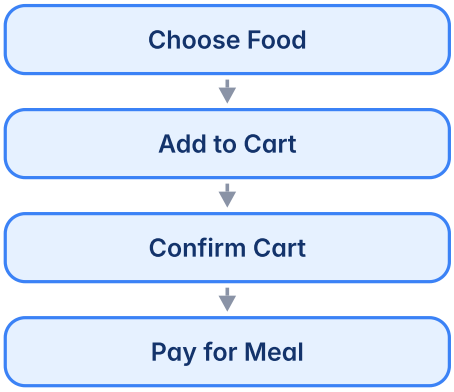


Figure 3-2: the rendered flowchart

Generating CSV and mock CSV data

CSV OUTPUT

```
name,age,grade
John,16,A
Alex,17,B
Emma,16,A
```

CSV = Comma-Separated Values. The comma separates the columns (name , age , grade → column column column). Say "Generate CSV" and GPT knows to use commas.

Mock CSV data = realistic but fake data generated for **testing** an app or demo — e.g. "Generate 5 rows of mock customer data as CSV with columns name, age, city." No real (private) data needed.

IMPORTANT LESSON ABOUT FORMATS

GPT **heavily depends on the format you specify**: "Give me CSV" → commas; "Give me JSON" → JSON syntax; "Give me Mermaid" → Mermaid syntax. Be very clear about the desired format.

Diffusion models — other modalities

A **diffusion model** is a type of generative AI commonly used to **create images** ("Create a picture of a cat wearing sunglasses" → diffusion model → image). When language models and image models work together we can move between forms of information — **Text → Image, Text → Code, Text → Diagram, Text → Story**. This is working with different **modalities/formats**.

UNIT II · SESSION 2 — TRANSFORMING, STYLING AND CHUNKING TEXT

2.10 Explain It Like I'm Five (ELI5)

KEY IDEA

ELI5 is a prompting technique: tell GPT *"Explain this like I'm 5 years old."* It means use very simple words and examples so even a beginner can understand.

DIFFICULT

```
Cancer involves abnormal
proliferation of neoplastic
cells...
```

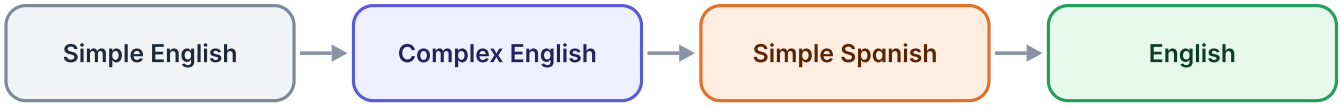
ELI5

```
Cancer is a sickness where some
cells in the body grow in a way
they shouldn't.
```

ELI5 is useful for: **technical documents · research papers · scientific concepts · difficult textbooks · complex programming topics**.

2.11 Universal Translation Through LLMs

LLMs can translate between languages — and between **levels of complexity**. The textbook example chains several transformations:



ENGLISH The sun rises and birds sing.

SPANISH El sol sale y los pájaros cantan.

SPANISH El sol sale y los pájaros cantan.

BACK The sun rises and the birds sing.

But translation isn't perfect

- **Meaning can be lost**: Original → Translation → Another transformation → Translation back — each step can slightly change the meaning.
- **High- vs low-resource languages**: LLMs perform better in languages with lots of training data; languages with fewer resources may give poorer translations.
- **Same for programming languages**: better at popular ones (Python, JavaScript, Java, C++); less reliable with very new languages, rare languages and new libraries/packages — because the model has seen more examples of popular technologies.

2.12 Ask for Context

KEY IDEA

When GPT doesn't have enough information, let it ask you for the context it needs before answering — instead of guessing.

BAD

```
Should I use MongoDB or PostgreSQL?
```

OUTPUT

```
"It depends on your requirements."
← not very useful
```

BETTER

```
Should I use MongoDB or PostgreSQL?

First, tell me what information you need to make a good recommendation.
```

GPT might ask you to consider: what type of data · is it structured · how much data · complex relationships? · how many users · performance needed · budget · high availability · security requirements · preferred technology. Then you provide it:

YOU

```
My application has:
- Structured customer data
- Many relationships between tables
- Complex queries
- 100,000 users
```

RESULT

```
Now GPT can give a much more informed recommendation (here: PostgreSQL).
```

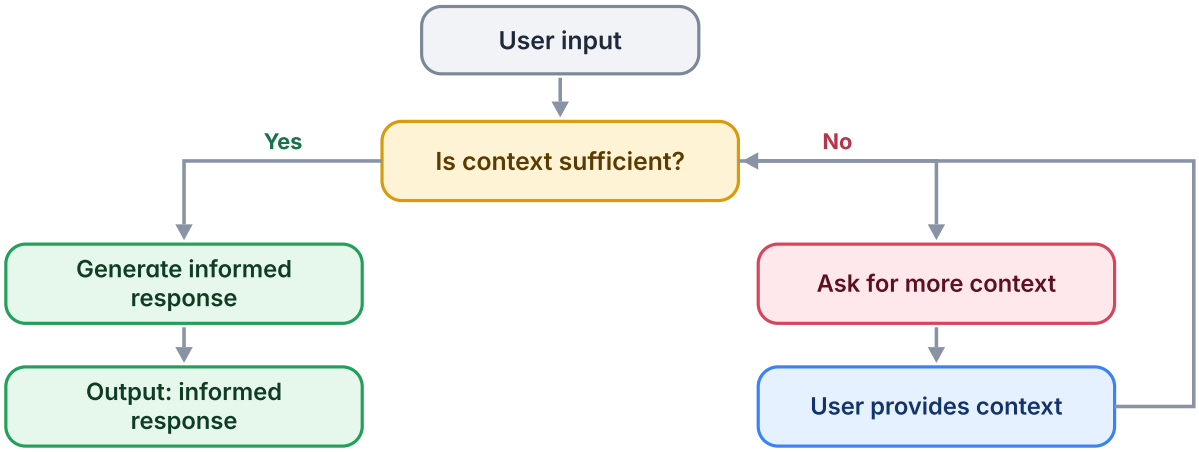


Figure 3-3: the decision process of an LLM while asking for context

EXAM TIP

Draw this loop diagram in the exam — "Is context sufficient? Yes → informed response; No → ask for more context → user provides context → check again."

2.13 Text Style Unbundling

DEFINITION

Text style unbundling = identifying the writing characteristics of a text (tone, length, vocabulary, structure, content) so that **new content can be created in a similar style**.

We look at a piece of writing and find out **how** it is written, separating its style into parts:

TEXT

```
AI is changing our world. It
creates exciting opportunities,
but we must use it responsibly.
```

Feature	Analysis
Tone	Positive + informative
Length	Short
Vocabulary	Simple
Structure	Intro → benefit → warning

Then we ask GPT to write another article in the **same style**.

Why is it useful?

Imagine a company with **100 writers** who all need to write in the same style. Style unbundling gives them a **style guide** so that everyone writes consistently.

1
Improved brand consistency
All company content sounds similar.

2
Faster content creation
Writers know exactly what style to follow.

3
Adaptability
Same style for blogs, emails, ads, social media, etc.

2.14 Identifying the Desired Textual Features

Before copying a writing style, decide **which parts of the style** you want to copy:

Feature	Meaning	Examples
1. Tone of voice	The feeling / attitude of the writing	Formal "We would like to inform you..." · Casual "Hey! Here's what you need to know." · Funny "Well, that didn't go as planned!" · Professional "The results demonstrate significant improvement."
2. Length	How long the new content should be	Short 100 words · Medium 500 words · Long 2000 words
3. Vocabulary & phrasing	What kind of words the writer uses	Simple "The system works quickly." · Technical "The system demonstrates high computational efficiency."
4. Structure	How the content is organised	Title → Introduction → Main points → Examples → Conclusion
5. Content	Which ideas the text usually focuses on	An AI article: explain AI → benefits → risks → ethical use

EXAM TIP

Memory hook for the 5 features: **"T-L-V-S-C"** → **T**one, **L**ength, **V**ocabulary, **S**tructure, **C**ontent ("**T**all **L**adies **V**isit **S**mall **C**afés").

2.15 Generating New Content with the Extracted Features

After identifying the style, use it to create new content. Suppose GPT's analysis is: Tone → informative and optimistic; Length → short; Vocabulary → simple; Structure → Problem → Benefits → Concerns → Conclusion. Now:

PROMPT
```
Write a new article about cybersecurity using the same tone, length, vocabulary, and structure.
```

OUTPUT
```
A short, optimistic, simply-worded article on cybersecurity that follows Problem → Benefits → Concerns → Conclusion.
```



Basic process of text style unbundling

2.16 Extracting Specific Textual Features with LLMs

We don't always need the whole style — we can ask GPT to extract specific information with a clear instruction.

PROMPT

```
Extract the entities from: "Artificial intelligence is transforming healthcare and entertainment."
```

OUTPUT

```
Entities:
- Artificial Intelligence
- Healthcare
- Entertainment
```

LLMs can identify: **names · organizations · locations · products · dates · topics · sentiment · tone · keywords**.

2.17 Chunking Text and Strategies

KEY IDEA

Chunking = breaking a large piece of text into smaller pieces (chunks). Book → chapters → paragraphs → sentences.

Why? LLMs have limits on how much text they can process at once (the **context length**). A huge document = too many tokens → the LLM may reject or cut off the request. Chunking solves this.

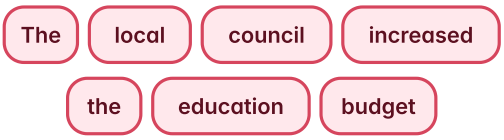
✓ WHEN TO USE CHUNKING

- **Large documents** — too big for the model (e.g. a 500-page book)
- **Complex analysis** — deep analysis of a large document
- **Multiple-topic documents** — AI / cybersecurity / cloud in one file → separate chunks

× WHEN NOT TO USE CHUNKING

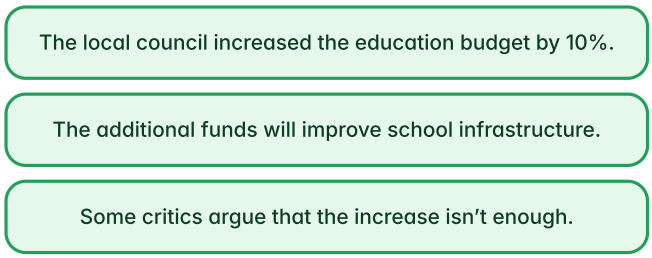
- **Short documents** — already small
- **Simple analysis** — don't make a simple task complicated
- **Single-topic documents** — small & focused, little benefit

× Poor chunking — split by word



meaning & relationships between words are lost
many tiny pieces → more processing

✓ Better chunking — split by sentence



Poor chunking causes loss of context and increased processing. Each good chunk contains meaningful information.

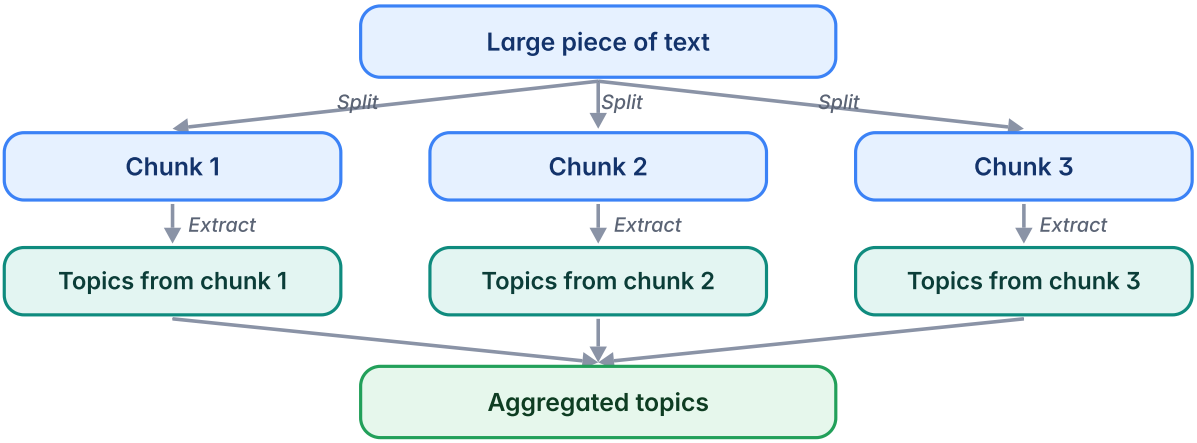


Figure 3-5: topic extraction with an LLM after chunking text

The six chunking strategies

Strategy	How	Advantages	Disadvantages	Useful for
① **By sentence**	Each sentence = a chunk	Preserves context; suitable for various tasks	May not be efficient for very long content	Summarization, translation, sentiment analysis
② **By paragraph**	Each paragraph = a chunk	Handles longer content; keeps related ideas together (cohesive units)	Less granularity; may miss subtle connections	Document analysis, topic analysis, information extraction
③ **By topic / section**	Group text by topic (AI → chunk 1, cyber → chunk 2)	Identifies main themes; better for classification	Requires topic identification; may miss fine details	Classification, recommendations, clustering
④ **By complexity**	Easy / medium / technical content	Groups similar complexity levels; adaptive	Requires complexity measurement; not for all tasks	Readability analysis, personalised learning, content adaptation
⑤ **By length**	Fixed size, e.g. 1000 words each	Manages very long content; efficient processing	Loss of context — a sentence/idea may be cut in the middle	Very large documents
⑥ **By tokens**	A tokenizer (e.g. **tiktoken**) splits by token count	Accurate token counts → avoids hitting prompt token limits	Requires tokenization; adds computational complexity	Staying within the model's token limit

COUNTING TOKENS WITH TIKTOKEN (OPENAI'S TOKENIZER)

```
import tiktoken

encoding = tiktoken.get_encoding("cl100k_base")   # tokenizer used by GPT-3.5 / GPT-4
tokens = encoding.encode("Chunking text is important.")
print(len(tokens))                                # how many tokens this text uses
```

EXAM TIP

Table 3-1 (six strategies with advantages & disadvantages) is a favourite 10-mark question. Memory hook: **"S-P-T-C-L-T"** → **S**entence, **P**aragraph, **T**opic, **C**omplexity, **L**ength, **T**okens.

2.18 Role Prompting

KEY IDEA

Role prompting = telling the AI to "act as" a particular type of person or expert — e.g. "Act as a teacher", "Act as a Python expert", "Act as a technical reviewer".

PLAIN

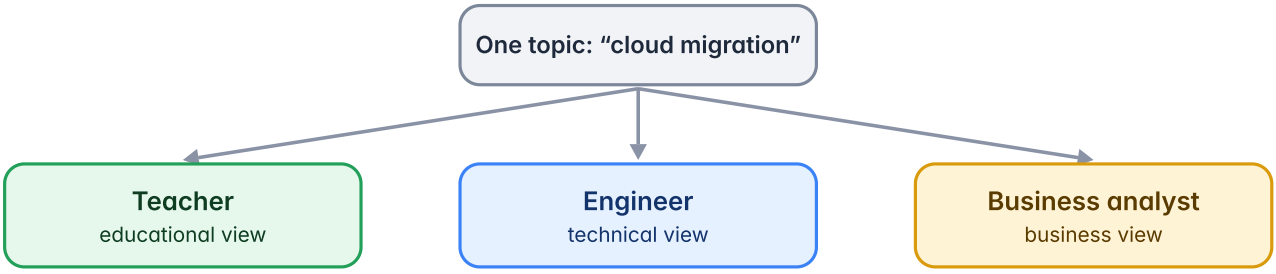
```
Review MongoDB.
```

ROLE

```
Act as a technology reviewer. Give
an in-depth review of MongoDB,
including features, advantages,
disadvantages, and comparisons.
```

Benefits of role prompting (6)

Benefit	Explanation / example
① **More focused answers**	The AI has a specific role: general GPT → broad response; tech reviewer → technical review.
② **Specific expertise**	"Act as a programming instructor" / "Act as a business analyst" — the role sets the context.
③ **Control response style**	Formal, casual, humorous, professional, educational — "Act as a friendly teacher and explain recursion."
④ **Encourage creativity**	"Act as a storyteller and explain the history of computers."
⑤ **Explore different perspectives**	Teacher → educational, engineer → technical, business analyst → business view.
⑥ **Improve user engagement**	Feels interactive — "Act as an interviewer and ask me Python interview questions."



Benefit ⑤: the same topic seen from different roles

Challenges of role prompting (3)

① Bias and stereotypes
Some roles carry stereotypes; assigning a role based on a stereotype can cause inappropriate or biased responses.

② Role drift
In a long conversation the AI may slowly stop following the role ("Act as a Python teacher" ... later talks about something unrelated).

③ Incorrect information
Calling the AI an "expert" doesn't make it correct. Role prompting changes the **style/context**, not the **reliability** of the facts.

When to use role prompting (5)

Use it when you need...	Example prompt	Good for
① **Specific expertise**	Act as a database expert.	Technical topics
② **A specific response style**	Act as a friendly teacher.	Learning
③ **Creative responses**	Act as a storyteller.	Stories, creative writing
④ **Different perspectives**	Analyze this problem from a software engineer's perspective.	Examining aspects of a topic
⑤ **More engaging conversations**	Act as an interviewer and conduct a mock interview.	Practice, interaction

2.19 GPT Prompting Tactics

This section collects techniques for getting better results. The goal: give the model better instructions and better information so the output is more useful and reliable.

Role prompting
"Act as ..." (2.18)

Text style unbundling
copy a writing style (2.13)

Asking for context
let GPT ask questions (2.12)

Reference text
answer from a given document (2.20)

Thinking time
solve → check → answer (2.21)

Task decomposition
break big tasks into steps (1.10)

2.20 Avoiding Hallucinations with Reference

KEY IDEA

A **hallucination** is when the model gives information that sounds correct but is actually unsupported or incorrect. Supplying **reference text** and telling GPT to answer **only** from it reduces hallucinations.

You give GPT an article that only says *"B2B customers usually have longer sales cycles."* and ask *"Are B2C sales more cost-effective?"* The article doesn't say that. Instead of guessing, instruct GPT:

PROMPT

```
Only answer using the provided article. If the answer isn't there, say "I could not find an answer."

Article: """B2B customers usually have longer sales cycles."""

Question: Are B2C sales more cost-effective?
```

OUTPUT

```
I could not find an answer.
```

How reference text works

PROMPT

```
Reference: A nuclear reactor is a device that initiates, controls and maintains a nuclear chain reaction.

Question: What is a nuclear reactor?
```

OUTPUT

```
A nuclear reactor is a device that controls and maintains a nuclear chain reaction.
```

Adding citations

PROMPT

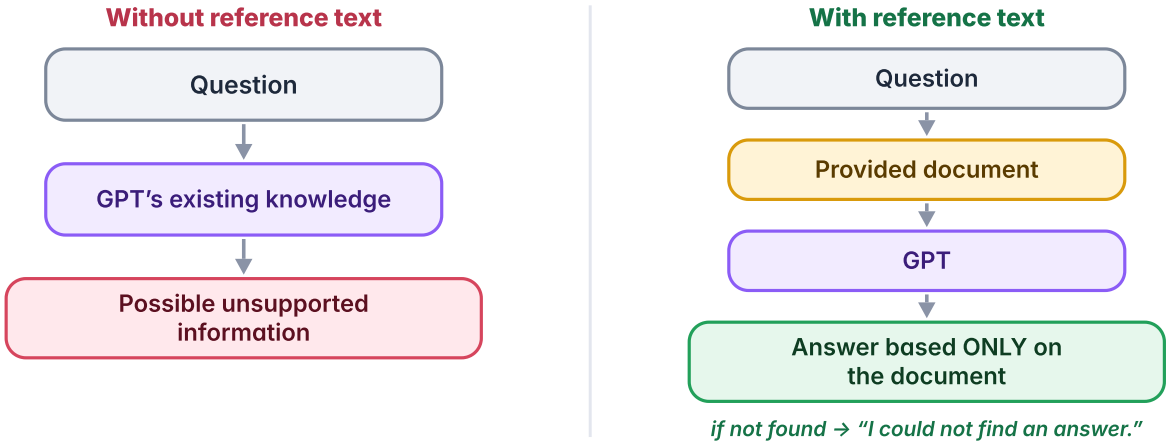
```
Answer only using the provided document and include the passage used as evidence.
```

OUTPUT

```
Answer: A nuclear reactor is a device that controls and maintains a nuclear chain reaction.

Citation: "A nuclear reactor is a device that initiates, controls, and maintains a nuclear chain reaction."
```

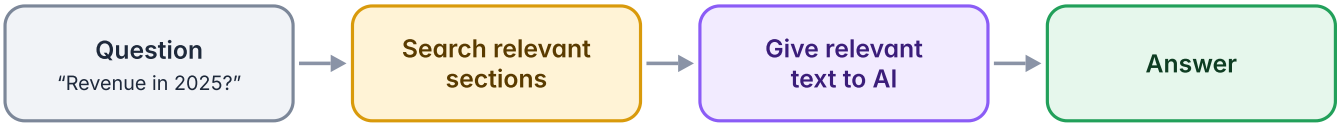
Citations make it easy to **check where the answer came from**.



Why reference text is useful: it gives you control over the source of the answer.

Dynamic retrieval of relevant information

GPT has a **limited context window**, so it can't always read an entire huge document. Instead: **store** the large document → **search** for parts relevant to the question → give **only those parts** to the AI → ask it to answer using them.



Example: a 500-page company report — send only the revenue section, not all 500 pages. This is Retrieval-Augmented Generation (RAG), covered fully in Unit IV.

2.21 Give GPT "Thinking Time" and Inner Monologue

KEY IDEA

Tell the model to work through the problem carefully before giving the final answer — e.g. "First solve the problem yourself. Then compare your answer with the proposed solution." The idea: **Solve → Check → Answer**.

PROMPT

```
Jane has 5 apples. John takes 2 apples from Mary. John gives those 2 apples to Jane.
How many apples does Jane have?

First work out your own solution step by step, then answer.
```

OUTPUT

```
Jane starts with 5.
John gets 2 from Mary.
John gives those 2 to Jane.
Jane now has 5 + 2 = 7.
```

The prompt makes the model check the problem carefully instead of immediately accepting a proposed answer.

Inner monologue tactic

Let the model do **intermediate work**, but don't show those intermediate steps to the user. In a tutoring app, the AI internally solves the problem, checks the student's answer, finds the mistake and creates a hint — the student sees **only the hint**.



Problem → Solve → Check student's answer → Create hint → Show hint

PROMPT

```
Student answer: Probability = 13/204  (drawing a Heart and a Club)
```

SHOWN

```
Hint: Remember that the two cards can be drawn in either order. Consider both Heart
→ Club and Club → Heart.
```

Why hide the reasoning? It gives guidance **without revealing the complete solution** — the student thinks and learns (e.g. "Hint: start by calculating the probability of getting a heart on the first draw.").

2.22 Self-Eval LLM Responses

KEY IDEA

Self-evaluation = asking the AI to review and improve its own previous answer: Generate → Review → Improve.

1ST PROMPT: "GENERATE A PYTHON HELLO WORLD FUNCTION."

```
def print_hello_world():
    print("Hello, World!")
```

2ND PROMPT: "CAN YOU IMPROVE IT? ADD TYPE HINTS OR DEFAULTS."

```
def print_message(message: str = "Hello,
World!") -> None:
    """Print the given message."""
    print(message)
```

Improvement	Meaning
Type hint `message: str`	`message` should normally be a string
Return type `-> None`	the function does not return a value
Default value `= "Hello, World!"`	`print_message()` prints "Hello, World!"; `print_message("Good morning!")` prints "Good morning!" → more flexible
Docstring `"""…"""`	documents what the function does

R2 Unit II — Quick Revision & Likely Questions

SESSION 1 IN 6 LINES

- Lists: control **count, filter, format, no commentary** + give an example list.
- Hierarchical list → parse with regex: `r'\*(.+)'`, `r'\s+[a-z]\. (.+)'`.
- Example 3-1 = two separate lists; Example 3-2 = dictionary (keeps structure).
- Avoid regex for complex data → ask for JSON / YAML.
- JSON: "only valid JSON" + example; `json.loads()`; handle preamble/invalid JSON.
- YAML: readable, fewer symbols, comments; filter against a schema → "No Items".

SESSION 2 IN 7 LINES

- ELI5 = explain simply.
- Translation chains lose meaning; low-resource languages weaker.
- Ask for context: "What info do you need?"
- Style unbundling: Tone, Length, Vocabulary, Structure, Content.
- Chunking: 6 strategies — sentence, paragraph, topic, complexity, length, tokens.
- Role prompting: 6 benefits, 3 challenges, 5 uses.
- Reference text + "I could not find an answer" → fewer hallucinations; add citations.

REGEX IN 4 LINES

- `\*` and `\.` = literal * and .
- `(.+)` = capture one-or-more characters.
- `\s*` vs `\s+` = zero-or-more vs one-or-more spaces.
- `findall` = all matches; `group(1)` = captured text.

COMMON MISTAKES

- Asking for 10 items ≠ getting 10 — validate.
- GPT output may be invalid JSON — use try/except.
- Word-level chunking destroys context.
- Role prompting doesn't make facts correct; watch for role drift.

Likely CIE questions — with answer plans

Explain hierarchical list generation and how the output is parsed using regular expressions (Example 3-1 / 3-2). — 10 marks

ANSWER PLAN

- Prompt with "hierarchical", "incredibly detailed" + example structure; output outline; tree diagram.
- Parsing definition.
- Explain both patterns token-by-token; `findall`; output.
- Limitation of 3-1 → dictionary approach in 3-2 with walkthrough table.

When should you avoid regular expressions? Explain generating JSON and the common problems. 8 marks

ANSWER PLAN

- Regex hard to maintain for complex data → structured formats.
- Prompt: only valid JSON + example; json.loads flow.
- Problems: preamble/code fences, invalid JSON → try/except.
- Regex vs JSON table.

What is YAML? Compare with JSON. Explain filtering YAML payloads. 8 marks

ANSWER PLAN

- Same data in JSON and YAML; 3 advantages.
- Schema of allowed items; user query; filtered YAML; "No Items".
- Figure 3-1 decision diagram; LLM as reasoning engine; still validate with code.

Write notes on diverse format generation: Mermaid, CSV, mock CSV data, ELI5, universal translation. 8 marks

ANSWER PLAN

- Mermaid code → flowchart.
- CSV columns; mock data for testing.
- ELI5 example (cancer).
- Translation chain + limitations.

What is "Ask for Context"? Explain with an example and the decision diagram. 5 marks

ANSWER PLAN

- MongoDB vs PostgreSQL bad/better prompts; list of questions GPT asks; Figure 3-3 loop.

Explain text style unbundling: identifying textual features and generating new content. 8 marks

ANSWER PLAN

- Definition + example analysis.
- Benefits: consistency, speed, adaptability.
- 5 features with examples.
- Process pipeline; cybersecurity example; extracting specific features (entities).

What is chunking? Explain the six chunking strategies with advantages and disadvantages. 10 marks

ANSWER PLAN

- Definition + why (context length).
- When to use / not use.
- Poor vs good chunking example.
- Table 3-1 (6 strategies) + tiktoken; topic extraction diagram.

Explain role prompting — benefits, challenges and when to use it. 8 marks

ANSWER PLAN

- Definition + MongoDB review example.
- 6 benefits, 3 challenges (bias, role drift, incorrect info), 5 use cases.

How can hallucinations be avoided? Explain reference text, citations, dynamic retrieval, thinking time and self-evaluation. 10 marks

ANSWER PLAN

- Hallucination definition; B2B example with “I could not find an answer”.
- Nuclear reactor + citations; with/without diagram.
- Dynamic retrieval (RAG preview).
- Thinking time (Jane’s apples), inner monologue (tutoring hint).
- Self-eval (Hello World with type hints/defaults).

UNIT III

Advanced Techniques for Text Generation with LangChain

CIE-1 covers the beginning of Unit III: why we need frameworks, what LangChain is, its modules, how to set it up, and how chat models work.

Why Basic Prompting Is Sometimes Not Enough 3.1

Introduction to LangChain 3.2

Six Common Modules of LangChain 3.3

Environment Setup for LangChain 3.4

Chat Models 3.5

Unit III (CIE-1 part) — Quick Revision & Likely Questions R3

⏱ ~45 minutes · Sitting 6

3.1 Why Basic Prompting Is Sometimes Not Enough

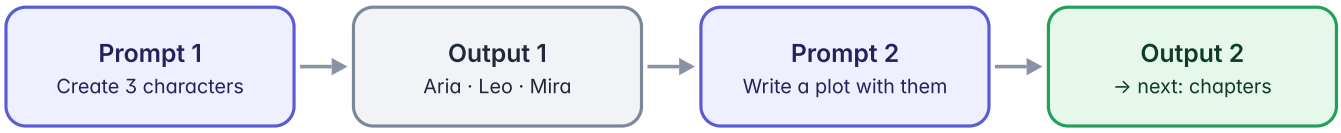
For real applications we often need more advanced Generative AI **techniques / frameworks**, for three reasons:

1. Context length

Context length = the amount of information an LLM can consider at one time. Ask "Summarise this entire 500-page book" — if the book is larger than the context window, you **cannot send it in one prompt**.

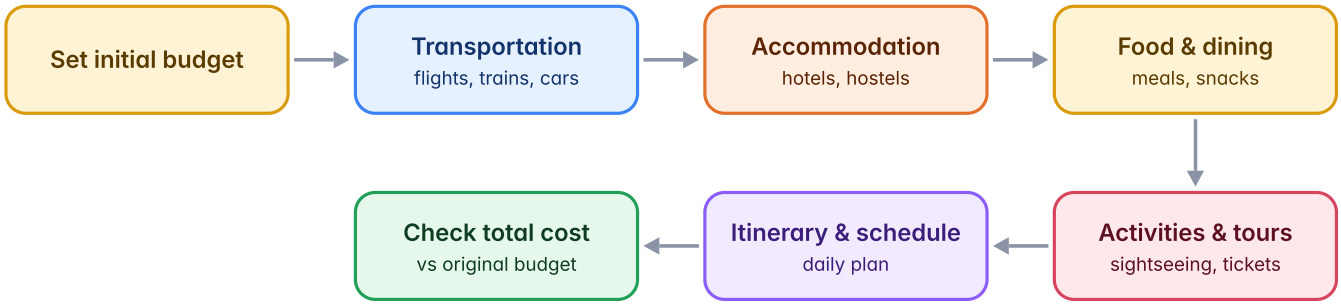
2. Combining sequential LLM inputs/outputs

Sometimes one LLM call isn't enough — the **output of one call becomes the input of the next**. This is a sequential LLM workflow, or chain.



3. Performing complex reasoning tasks

"Plan a trip for five days with a budget of ₹50,000, considering transportation, hotels, food and activities." The LLM must consider several things — an advanced system breaks it into smaller tasks and solves them one by one (**task decomposition**).



The trip-planning problem decomposed into sub-tasks

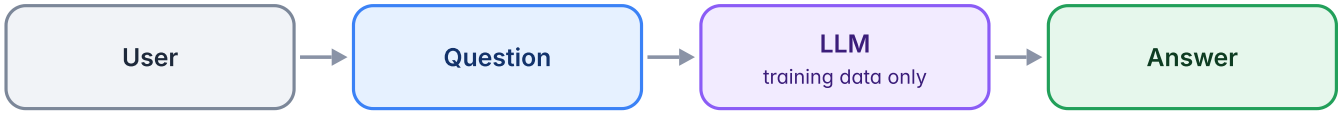
3.2 Introduction to LangChain

KEY IDEA

LangChain is a framework used to build applications that use Large Language Models (LLMs) such as GPT. It is available for both **Python** and **TypeScript**.

Need for LangChain

An LLM by itself mainly works like this:



But real-world applications often need more than just asking questions. We may want an AI system to:

- Read information from a **database**
- **Search** documents
- **Remember** previous conversations
- Use external **tools**
- Perform **multiple steps**
- Make **decisions**
- Take **actions**

REMEMBER

LangChain helps us connect an LLM with these additional components.

The two main ideas of LangChain

1. Enhance data awareness

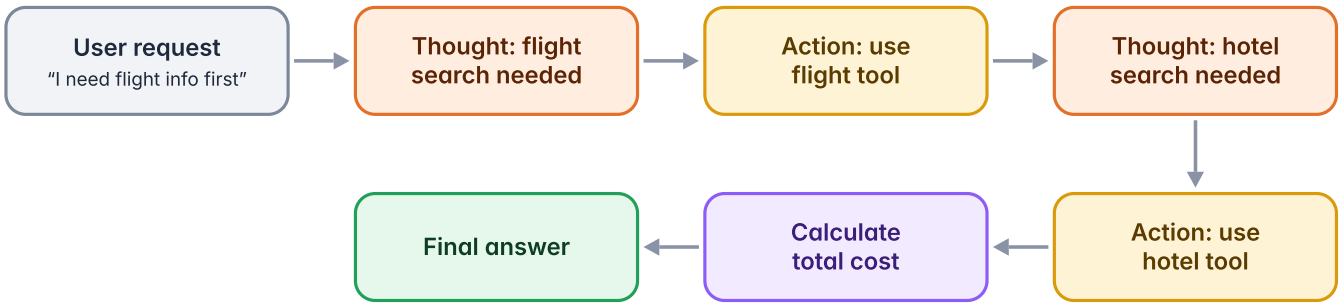
An LLM may not know about your **private or external data** (e.g. employee information in a company database). LangChain connects the LLM to that data source.



LangChain connects the LLM with external data sources.

2. Enhance agency

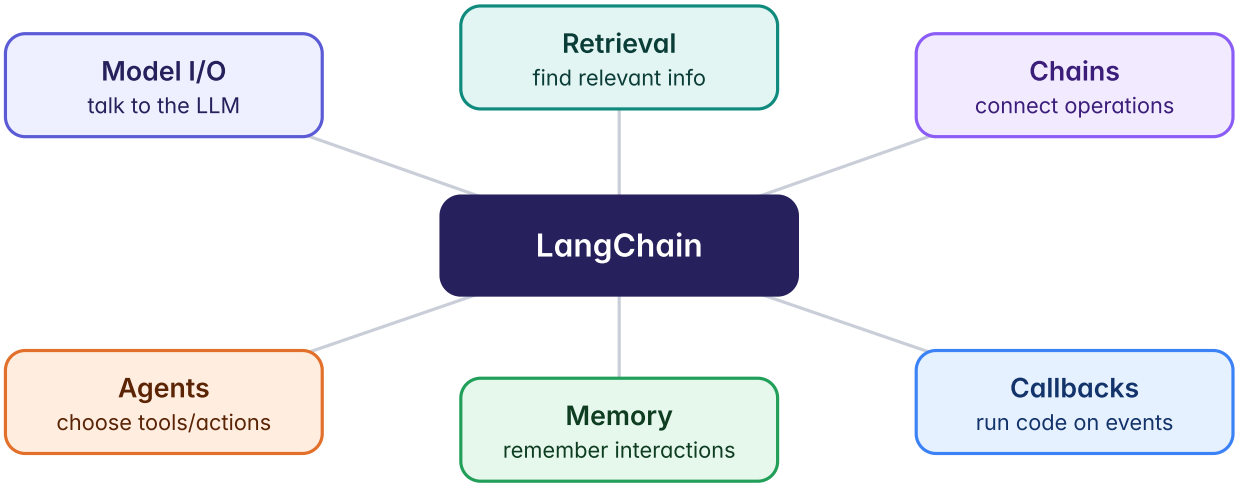
Agency = giving the LLM the ability to decide what action to take and use tools to accomplish a goal. An AI travel assistant might need to: 1) search flights, 2) check hotels, 3) calculate costs, 4) create an itinerary. Instead of simply answering, the LLM decides which tool to use next.



LangChain helps LLMs interact with their environment and use tools (this becomes "agents" in Unit IV).

3.3 Six Common Modules of LangChain

LangChain provides reusable modules — think of them as **building blocks** for AI applications.



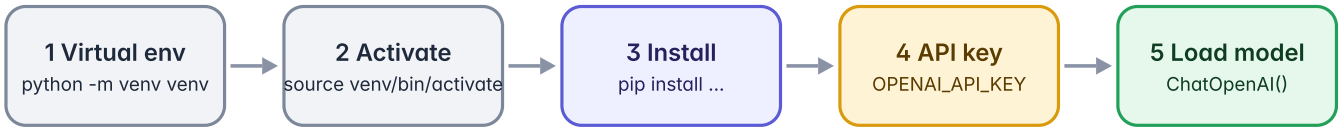
Module	Simple meaning
Model I/O	Communicating with the LLM (prompts in, outputs out)
Retrieval	Finding relevant information (documents, databases)
Chains	Connecting multiple operations in sequence
Agents	Deciding which tool / action to use
Memory	Remembering information between interactions
Callbacks	Running code when specific events happen (logging, streaming, monitoring)

EXAM TIP

Memory hook for the six modules: **"My Rabbit Can Always Munch Carrots"** → **M**odel I/O, **R**etrieval, **C**hains, **A**gents, **M**emory, **C**allbacks.

3.4 Environment Setup for LangChain

Before using LangChain: **install it**, create a **Python environment**, install required packages, and connect to an **LLM provider** such as OpenAI.



1. Install LangChain

USING PIP

```
pip install langchain langchain-openai
```

USING CONDA

```
conda install -c conda-forge langchain
langchain-openai
```

Package	Purpose
`langchain`	the main LangChain framework
`langchain-openai`	lets LangChain communicate with OpenAI models
`openai`	OpenAI's own Python package (`pip install openai`)

2. Use a virtual environment (recommended)

TERMINAL

```
# Create a virtual environment
python -m venv venv

# Activate the virtual environment
source venv/bin/activate        # on Windows: venv\Scripts\activate

# Install the dependencies for the whole book
pip install -r requirements.txt
```

If you prefer to install all package requirements for the entire book, use the `requirements.txt` file from the book's GitHub repository.

3. Connect to a model provider (API key)

LangChain needs integrations with one or more **model providers**. For OpenAI, it is best practice to set an environment variable called `OPENAI_API_KEY` in your terminal, or load it from a `.env` file using `python-dotenv`.

.ENV FILE

```
# .env  (never commit this file to Git!)
OPENAI_API_KEY=sk-...your-key...
```

PYTHON

```
from dotenv import load_dotenv
load_dotenv()          # reads .env into environment variables

from langchain_openai.chat_models import ChatOpenAI
chat = ChatOpenAI()    # picks up OPENAI_API_KEY automatically
```

For quick **prototyping** only, you can skip this and pass the key directly when loading a chat model:

PROTOTYPING SHORTCUT

```
from langchain_openai.chat_models import ChatOpenAI
chat = ChatOpenAI(api_key="api_key")
```

WATCH OUT

Hard-coding the API key in code is risky — anyone who sees the code (e.g. on GitHub) can use your key and run up charges. Use environment variables / `.env` for real projects.

3.5 Chat Models

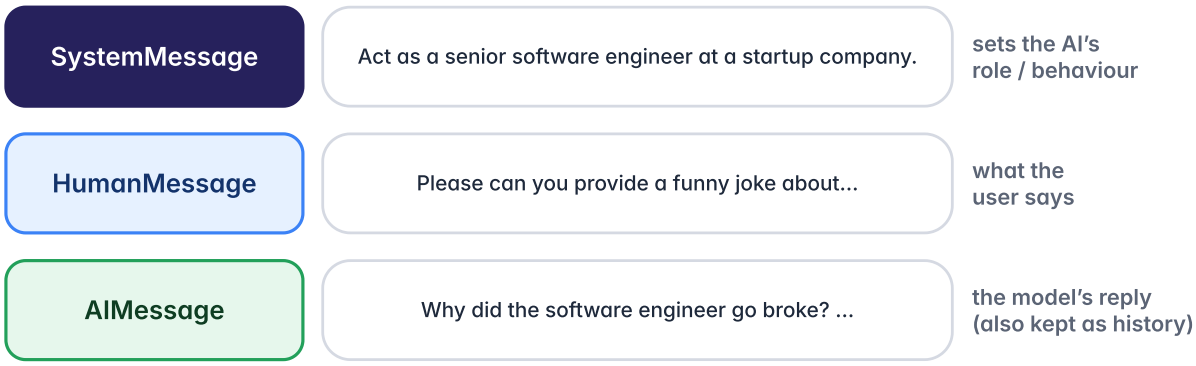
FROM THE TEXTBOOK

Your slides stop at environment setup, but "Chat Models" is in your CIE-1 portion. This section is taken from the textbook (Chapter 4) so you are fully covered.

KEY IDEA

A **chat model** (like OpenAI's GPT-4) takes a list of messages as input and returns a **message** as output. LangChain gives every provider's chat model the **same simple interface**, so you can swap models easily.

The three message types



Message type	Who it represents	Purpose
`SystemMessage`	the developer / system	Sets the AI's **role, persona or rules** ("Act as a senior software engineer...") — this is **role prompting**
`HumanMessage`	the user	The user's question or instruction
`AIMessage`	the AI	The model's reply — stored to keep **conversation history**

TEXTBOOK EXAMPLE — USING A CHAT MODEL IN LANGCHAIN

```
from langchain_openai.chat_models import ChatOpenAI
from langchain.schema import AIMessage, HumanMessage, SystemMessage

chat = ChatOpenAI(temperature=0.5)          # 0.5 = balanced creativity

messages = [
    SystemMessage(content="""Act as a senior software engineer
at a startup company."""),
    HumanMessage(content="""Please can you provide a funny joke
about software engineers?"""),
]

response = chat.invoke(input=messages)       # send the messages to the model
print(response.content)                      # response is an AIMessage
```

OUTPUT

```
Sure, here's a lighthearted joke for you:
Why did the software engineer go broke?
Because he lost his domain in a bet and couldn't afford to renew it.
```

Line-by-line

Code	What it does
`ChatOpenAI(temperature=0.5)`	Creates the chat model object (uses `OPENAI_API_KEY`). **temperature** controls randomness (Unit I, 1.3).
`SystemMessage(...)`	Gives the model its **role**.
`HumanMessage(...)`	The user's actual request.
`chat.invoke(input=messages)`	Sends the whole list of messages to the model and waits for the reply.
`response.content`	The text of the returned `AIMessage`.

NOTE

In newer LangChain versions the messages are imported from `langchain_core.messages` instead of `langchain.schema`; the code is otherwise the same.

REMEMBER

To continue a conversation, append the reply (`AIMessage`) and the next `HumanMessage` to the list and call `invoke()` again — the model sees the **whole history** each time.

WHAT COMES AFTER

Coming next (CIE-2): **streaming chat models**, **prompt templates**, **LCEL** (LangChain Expression Language) and using prompt templates with chat models.

UNIT III · REVISE

R3 Unit III (CIE-1 part) — Quick Revision & Likely Questions

UNIT III IN 5 LINES

- Basic prompting isn't enough because of **context length**, the need to **chain** LLM calls, and **complex multi-step reasoning**.
- LangChain = framework (Python & TypeScript) to build LLM apps; two ideas: **data awareness** and **agency**.
- Six modules: Model I/O, Retrieval, Chains, Agents, Memory, Callbacks.
- Setup: venv → pip install langchain langchain-openai → OPENAI_API_KEY (.env + python-dotenv) → ChatOpenAI().
- Chat models take a list of messages: SystemMessage, HumanMessage, AIMessage; call `invoke()`; read `.content`.

What is LangChain? Why is it needed? Explain its two main ideas and six modules. 10 marks

ANSWER PLAN

- Definition; LLM alone (User → Question → LLM → Answer) vs real-world needs.
- Data awareness (DB flow) and agency (travel assistant flow).
- Hub diagram + module table.

Explain the environment setup for LangChain. 5 marks

ANSWER PLAN

- pip/conda install; venv commands; requirements.txt.
- Provider package; OPENAI_API_KEY via env/.env + python-dotenv; api_key shortcut for prototyping (risk).

What are chat models in LangChain? Explain message types with a code example. 6 marks

ANSWER PLAN

- List of messages in → message out; same interface across providers.
- System/Human/AI message table.
- ChatOpenAI code + invoke + response.content; temperature.

LAST-MINUTE REVISION

★ One-Page Cheat Sheet

FORMULAS

Prompt → AI model → Output
Better prompt → better output
Good Prompt = Clear Task + Style + Format + Examples + Constraints

5 PRINCIPLES

Direction (WHAT) · **F**ormat (HOW) · **E**xamples (SHOW) · **E**valuate (CHECK) · **D**ivide (BREAK)

LLM PIPELINE

Text → Tokens → IDs → Embeddings → Transformer (self-attention) → probabilities → next token → repeat

DECISION MECHANISM

w_{next} = arg max$_w$ P(w | $w_1 ... w_m$) · low temp = safe · high temp = creative

EXAMPLES

Zero-shot (0) · One-shot (1) · Few-shot (2–3 diverse) · too many → copying

EVALUATION

Two prompts → 5 runs each → DataFrame → CSV → shuffle (blind) → rate 1/0 → score

11 EVAL CRITERIA

Cost · Latency · Calls · Performance · Classification · Reasoning · Hallucination · Safety · Refusals · Adversarial · Similarity (BLEU/ROUGE)

CHATGPT TRAINING

Demonstrations → SFT → Reward model + PPO = **RLHF** (helpful, harmless, aligned)

ANALOGIES

Quantization 3.14 · LoRA adapters · MoE hospital · context window = desk · attention = classroom/voting

REGEX

`\* (.+)` headings · `\s+[a-z]\. (.+)` sub-headings · `findall` · `group(1)` · dict keeps structure

JSON / YAML

"Only return valid JSON" + example · `json.loads` · try/except · YAML = readable + comments · filter against schema → "No Items"

STYLE FEATURES

Tone · Length · Vocabulary · Structure · Content (T-L-V-S-C)

6 CHUNKING STRATEGIES

Sentence · Paragraph · Topic · Complexity · Length · Tokens (tiktoken)

ROLE PROMPTING

Benefits: focus, expertise, style, creativity, perspectives, engagement · Challenges: bias, role drift, wrong info

ANTI-HALLUCINATION

Answer ONLY from reference · "I could not find an answer" · cite the passage · retrieve relevant chunks (RAG) · thinking time · self-eval

LANGCHAIN

LangChain: data awareness + agency · Model I/O, Retrieval, Chains, Agents, Memory, Callbacks · System/Human/AI messages · `chat.invoke()`

LOOK UP ANY TERM

A–Z Glossary

A/B testing — comparing two prompt versions by running and rating both.

Adversarial testing — deliberately trying to break a prompt/system (e.g. prompt injection).

Agency — an LLM's ability to decide actions and use tools to reach a goal.

Agent — an AI system that plans, acts with tools, observes and repeats.

AI chaining — connecting several AI steps so each output feeds the next.

Attention / self-attention — mechanism letting each token weigh how important every other token is.

BERT — encoder-only Transformer used for understanding text.

BLEU / ROUGE — scores measuring how similar an AI answer is to a reference answer.

Blind evaluation — rating shuffled responses without knowing which prompt produced them.

Blind prompting — trial-and-error prompting judged only by "looks good".

BPE — Byte Pair Encoding — tokenizer that merges frequent character pairs.

Chunking — splitting large text into smaller meaningful pieces.

Context window — maximum tokens a model can consider at once.

CSV — Comma-Separated Values — rows of data with commas between columns.

Decoder — Transformer part that generates output token by token (GPT).

Diffusion model — generative model commonly used to create images.

ELI5 — "Explain it like I'm five" — simplest possible explanation.

Embedding — vector capturing a word's meaning and relationships.

Encoder — Transformer part that builds an understanding of the input (BERT).

Eval — systematic measurement of how well a prompt/model performs.

Few-shot — prompt containing several examples.

Fine-tuning — extra training of a pretrained model for a specific purpose.

GPU — processor doing many calculations in parallel; speeds up AI.

Hallucination — confident but unsupported or false output.

Inner monologue — model reasons internally; only the final part is shown.

JSON — structured key–value data format; parsed with json.loads().

LangChain — framework (Python/TypeScript) for building LLM applications.

Latency — time the model takes to respond.

LLM — Large Language Model trained on huge text to understand/generate language.

LoRA — Low-Rank Adaptation — fine-tune with small trainable adapters.

Mermaid — text language that renders into diagrams/flowcharts.

Meta prompting — using AI to write a prompt for another AI.

Mixture of Experts — model whose router activates only selected expert sub-networks.

Mock data — realistic fake data generated for testing.

Multimodal — works with several types of input: text, image, audio.

One-shot — prompt with exactly one example.

Parameters — learned weights of a neural network.

Parsing — converting unstructured text into structured data.

Persona — a person/style the AI is told to imitate (Steve Jobs style).

Pretraining — first training stage on huge general text.

Progressive summarization — summarise chunks, then summarise the summaries.

Prompt injection — malicious text like "Ignore your previous instructions…".

PPO — Proximal Policy Optimization — RL algorithm used in RLHF.

Quantization — storing parameters with lower numeric precision.

RAG — Retrieval-Augmented Generation — retrieve relevant text, then answer.

Regex — pattern for finding specific text (Python module re).

Reward model — model trained on human preferences to score responses.

RLHF — Reinforcement Learning from Human Feedback.

Role drift — AI gradually stops following its assigned role.

Role prompting — "Act as ..." instruction to set expertise/style.

Schema — definition of allowed structure/items (e.g. YAML filter).

SFT — Supervised fine-tuning on human-written example answers.

Style unbundling — extracting tone, length, vocabulary, structure, content of a text.

Temperature — setting controlling randomness of token choice.

Tensor — multi-dimensional grid of numbers (rank = number of axes).

tiktoken — OpenAI's tokenizer library for counting tokens.

Token / tokenization — basic text unit / splitting text into tokens.

Transformer — 2017 architecture built on attention ("Attention Is All You Need").

YAML — indentation-based, human-readable structured data format with comments.

Zero-shot — prompt with no examples.

SCORE MORE WITH THE SAME KNOWLEDGE

Exam-Day Answer-Writing Tips

Start with a 1-line definition

Use the **Key idea** box wording — examiners look for the definition first.

Always draw a diagram

Flow charts from this guide (pipeline, RLHF, chunking, YAML filter, ask-for-context) earn easy marks.

Give one prompt example

Write a bad prompt vs a good prompt in two boxes — it proves understanding instantly.

Use tables for "compare"

BERT vs GPT, JSON vs YAML, Regex vs JSON, Direction vs Format, chunking strategies.

Match marks to points

5 marks ≈ 5 points + 1 diagram; 10 marks ≈ definition + 8–10 points + diagram + example + conclusion.

End with "why it matters"

One closing line: reliability, consistency, lower cost, fewer hallucinations.